Hướng dẫn thực hành: Ứng dụng Báo thức có Mini Game (Android – Java)

Oct 13, 2026 · @Mai Ha

1. Mục tiêu và yêu cầu

Sau buổi thực hành, sinh viên xây dựng được ứng dụng **SmartAlarm**: đặt nhiều báo thức, chọn chuông, chọn kiểu lặp, và khi chuông reo phải giải đúng một mini game mới tắt được. Bài thực hành vận dụng trực tiếp ba nội dung của Bài 3: Service, Notification và Broadcast Receiver.

Môi trường: Android Studio (bản mới nhất), ngôn ngữ Java, minSdk 26 (Android 8.0), targetSdk 34. Thời lượng gợi ý: 2 buổi (6 tiết).

1.1. Yêu cầu chức năng

1. Hiển thị danh sách báo thức; bật/tắt từng báo thức bằng công tắc; chạm để sửa, nhấn giữ để xóa.
2. Thêm/sửa báo thức: chọn giờ, nhãn, kiểu chuông, rung, loại mini game và số câu phải giải.
3. Bốn kiểu lặp: **Một lần**, **Hằng ngày**, **Hằng tuần** (một thứ cố định mỗi tuần), **Tùy chọn các ngày trong tuần** (ví dụ T2–T6).
4. Đến giờ: phát chuông lặp lại + rung, hiện Notification và màn hình báo thức toàn màn hình (kể cả khi khóa máy).
5. Muốn tắt chuông phải hoàn thành mini game: **Giải toán** (trả lời đúng N phép tính) hoặc **Bấm theo thứ tự** (chạm các ô 1→9 đúng thứ tự). Nút Back bị vô hiệu hóa.
6. Báo thức lặp được tự động đặt lại cho lần kế tiếp; khởi động lại máy vẫn giữ báo thức.

1.2. Kiến thức áp dụng từ slide bài giảng

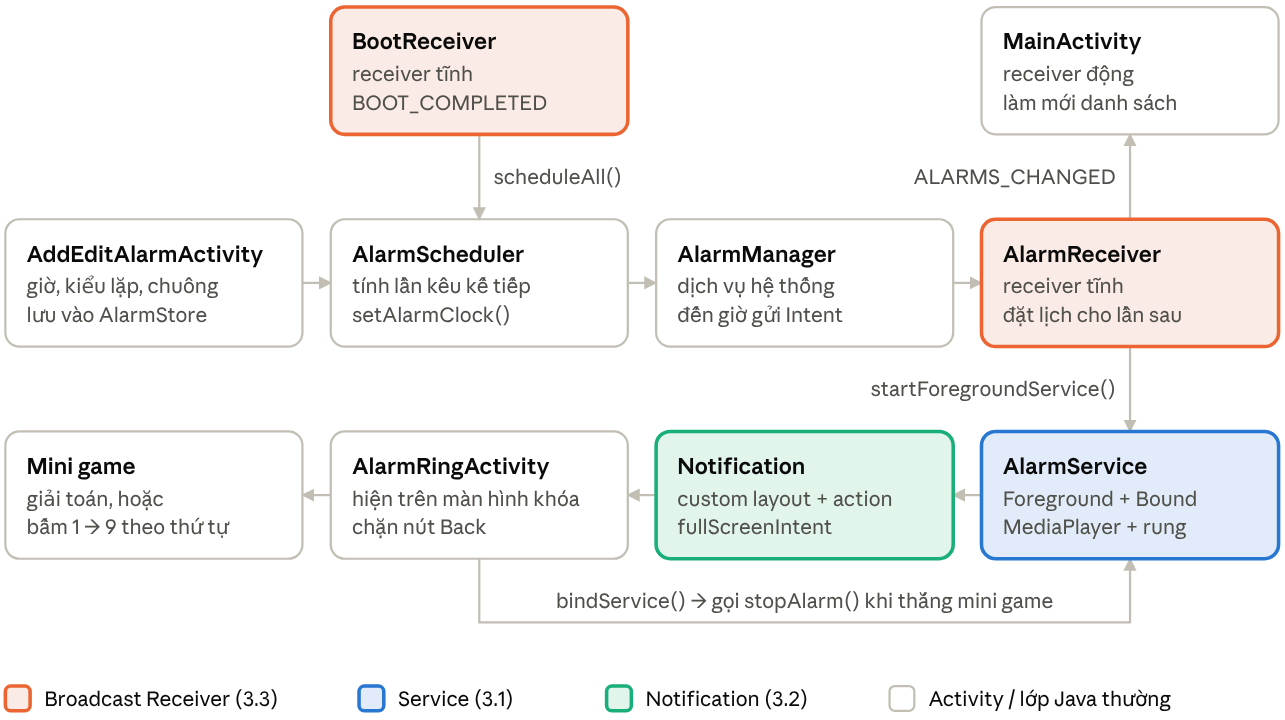
Nội dung trong slide	Áp dụng trong SmartAlarm
3.1 Foreground Service	`AlarmService` phát chuông + rung, luôn kèm Notification nên khó bị hệ thống dừng

Nội dung trong slide	Áp dụng trong SmartAlarm
3.1 Bound Service (`bindService` , `IBinder`)	`AlarmRingActivity` bind vào `AlarmService` để gọi `stopAlarm()` khi thắng mini game
3.1 Vòng đời Service (`onCreate` , `onStartCommand` , `onBind` , `onDestroy`)	Khởi tạo/giải phóng `MediaPlayer` , `Vibrator` đúng chỗ
3.2 Custom Layout Notification	Notification báo thức dùng layout riêng `notification_alarm.xml` (RemoteViews)
3.2 Action Notification + Full-screen Intent	Nút “Giải mini game” trong Notification, mở `AlarmRingActivity` khi đang khóa máy
3.3 Broadcast Receiver khai báo tĩnh	`AlarmReceiver` nhận Intent từ `AlarmManager` ; `BootReceiver` nhận `BOOT_COMPLETED`
3.3 Broadcast Receiver khai báo động	`AlarmRingActivity` đăng ký receiver trong `onStart()` , gỡ trong `onStop()`
3.3 Bảo mật Broadcast (permission)	Broadcast nội bộ dùng `setPackage()` và custom permission, `exported="false"`
Activity & Intent (bài trước)	Truyền `alarmId` giữa các màn hình bằng Intent extras

Ghi chú: dữ liệu báo thức lưu bằng `SharedPreferences` + JSON cho gọn; bài “Lưu trữ / SQLite” sau có thể thay bằng SQLite (xem Bài tập mở rộng).

2. Kiến trúc ứng dụng

Ứng dụng gồm 3 Activity, 1 Service, 2 Broadcast Receiver và 4 lớp Java hỗ trợ. Khi đến giờ, hệ thống gửi Intent tới `AlarmReceiver` ; receiver khởi động `AlarmService` ; service phát chuông và hiện Notification mở màn hình mini game; thắng mini game thì Activity gọi `stopAlarm()` qua Bound Service.



Luồng xử lý khi đến giờ báo thức · 10 thành phần

Hàng trên là lúc đặt báo thức, hàng dưới là lúc chuông reo. Màu cho biết thành phần thuộc mục nào của slide.

2.1. Danh sách file cần tạo

File	Loại	Vai trò
`Alarm.java`	Model	Dữ liệu một báo thức; tính thời điểm kêu kế tiếp
`AlarmStore.java`	Lưu trữ	Đọc/ghi danh sách báo thức (SharedPreferences + JSON)
`AlarmScheduler.java`	Tiện ích	Đặt/hủy lịch với `AlarmManager`
`AlarmReceiver.java`	Broadcast Receiver (tĩnh)	Nhận Intent đến giờ, khởi động Service
`BootReceiver.java`	Broadcast Receiver (tĩnh)	Đặt lại báo thức sau khi khởi động máy

File	Loại	Vai trò
`AlarmService.java`	Foreground + Bound Service	Phát chuông, rung, hiện Notification
`MathQuestion.java`	Tiện ích	Sinh phép toán ngẫu nhiên + 4 đáp án
`MainActivity.java` + `AlarmAdapter.java`	Activity	Màn hình 1: danh sách báo thức
`AddEditAlarmActivity.java`	Activity	Màn hình 2: thêm/sửa báo thức, chọn chuông
`AlarmRingActivity.java`	Activity	Màn hình 3: báo thức đang kêu + mini game
`res/layout/*.xml` (5 file)	Giao diện	`activity_main`, `item_alarm`, `activity_add_edit_alarm`, `activity_alarm_ring`, `notification_alarm`

2.2. Quy ước lưu ngày trong tuần

Mỗi báo thức lưu một số nguyên `daysMask` 7 bit, bit 0 = Thứ Hai … bit 6 = Chủ Nhật. Cả bốn kiểu lặp đều quy về cùng một thuật toán tìm ngày kế tiếp:

Kiểu lặp	`repeatType`	`daysMask`	Ví dụ
Một lần	0	`0`	Kêu một lần rồi tự tắt
Hằng ngày	1	`0x7F` (1111111)	Mọi ngày
Hằng tuần	2	một bit, ví dụ `1 << 0`	Mỗi Thứ Hai
Tùy chọn ngày	3	nhiều bit, ví dụ `0x1F` (0011111)	T2–T6

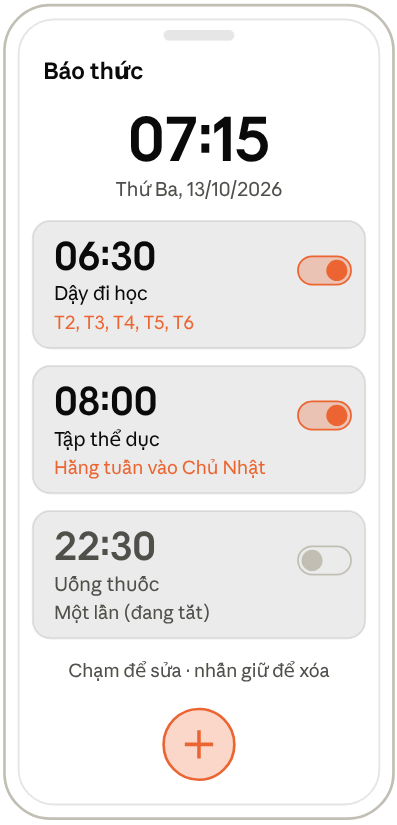
`Calendar.DAY_OF_WEEK` đánh số CN = 1, T2 = 2 … T7 = 7, nên đổi sang chỉ số 0..6 bằng công thức `(dayOfWeek + 5) % 7`.

3. Thiết kế giao diện (GUI)

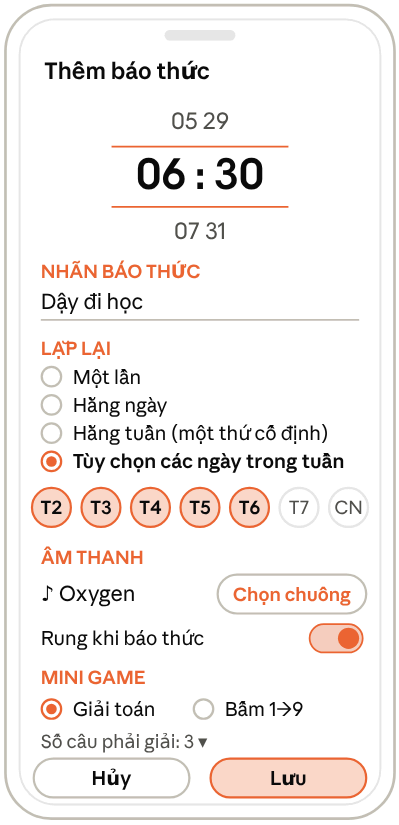
Ứng dụng có 3 màn hình, 1 hộp thoại chọn chuông và 1 Notification tùy biến. Mockup dưới đây là bố cục mục tiêu; layout XML đầy đủ nằm ở Bước 3 của phần 4.

SmartAlarm có 3 màn hình và 1 Notification

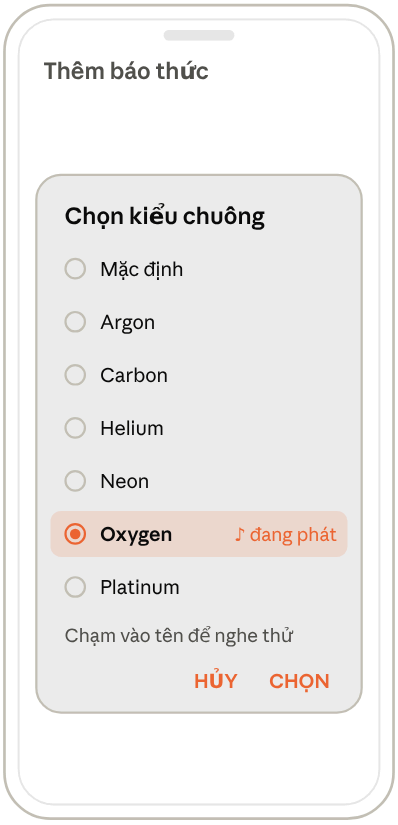
Mockup bố cục; màu cam là màu nhấn (accent #FF6B35) của ứng dụng



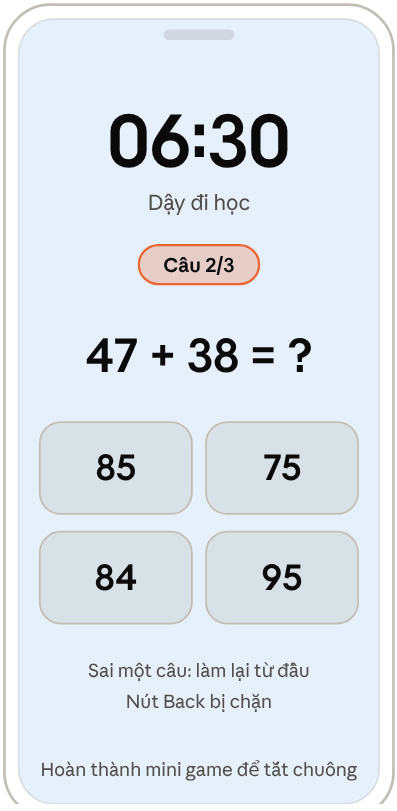
① Danh sách báo thức

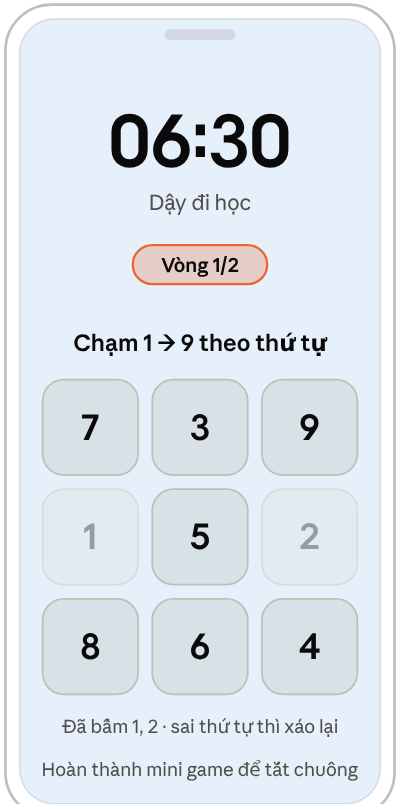


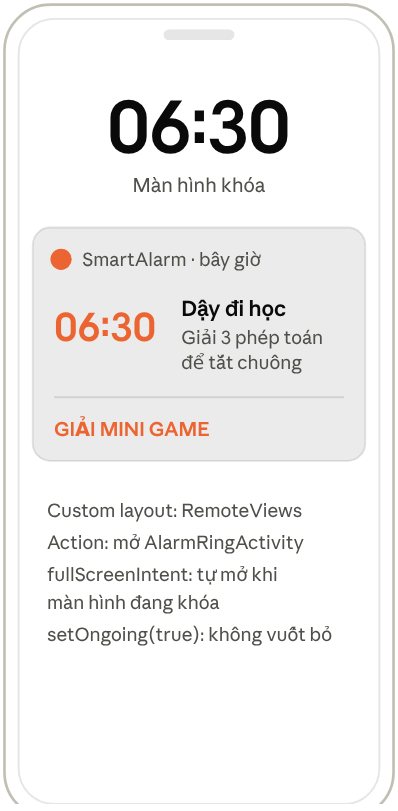
② Thêm / sửa báo thức



②b Hộp thoại chọn chuông









③a Mini game Giải toán **③b Mini game Bấm 1→9** **④ Notification khi chuông reo**

Mockup giao diện · 3 màn hình + hộp thoại + Notification

3.1. Đặc tả từng màn hình

Màn hình	Layout	Thành phần giao diện chính	Thao tác
① Danh sách báo thức (`MainActivity`)	`activity_main.xml`, `item_alarm.xml`	`TextClock` giờ/ngày hiện tại; `RecyclerView` gồm các `MaterialCardView` (giờ, nhãn, kiểu lặp, chuông + game, `SwitchCompat`); `FloatingActionButton` "+"; `TextView` báo rỗng	Chạm thẻ → sửa; nhấn giữ → hộp thoại xóa; gạt công tắc → bật/tắt và Toast "Báo thức sẽ kêu sau …"
② Thêm/sửa báo thức (`AddEditAlarmActivity`)	`activity_add_edit_alarm.xml`	`TimePicker` (spinner, 24h); `EditText` nhãn; `RadioGroup` 4 kiểu lặp; `Spinner` chọn thứ (Hằng tuần); 7 `CheckBox` hình tròn T2–CN (Tùy chọn); `TextView` + nút "Chọn chuông"; `SwitchCompat` rung; `RadioGroup` loại game; `Spinner` số câu; nút Hủy/Lưu	Chọn kiểu lặp thì chỉ hiện vùng chọn ngày tương ứng; Lưu → lưu + đặt lịch + Toast
②b Hộp thoại chọn chuông	`AlertDialog.setSingleChoiceItems`	Danh sách âm báo thức của hệ thống (`RingtoneManager.TYPE_ALARM`) + "Mặc định"	Chạm → nghe thử; CHỌN → lưu URI và tên chuông
③ Báo thức đang kêu (`AlarmRingActivity`)	`activity_alarm_ring.xml`	Nền tối; `TextClock` lớn; nhãn; nhãn tiến độ "Câu 2/3"; câu hỏi; `panelMath` 4 nút đáp án 2×2 hoặc `panelSequence` lưới `GridLayout` 3×3	Hiện trên màn hình khóa; chặn nút Back; hoàn thành đủ số câu/vòng → tắt chuông và đóng
④ Notification	`notification_alarm.xml`	`RemoteViews` gồm giờ (màu cam), nhãn, dòng nhắc mini game; nút action "Giải mini game"	Không vuốt bỏ được; chạm hoặc bấm action → mở màn hình ③

3.2. Quy ước giao diện

- Màu nhấn `#FF6B35` (cam) cho nút chính, kiểu lặp, ngày được chọn; nền sáng `#F4F6FB` cho màn hình ① ②; nền tối `#14182B` cho màn hình ③ để không chói mắt khi vừa thức dậy.
- Nút trong mini game cao 72–88dp, chữ 24–28sp, dễ bấm khi còn ngái ngủ.
- Báo thức đang tắt hiển thị mờ (alpha 0.45) để phân biệt với báo thức đang bật.
- Các style dùng chung (`SectionTitle`, `DayCircle`, `AnswerButton`, `SequenceButton`) khai báo trong `styles.xml` để layout gọn.

4. Các bước thực hành

Làm lần lượt 11 bước; sau Bước 11 ứng dụng chạy được hoàn chỉnh. Tất cả lớp Java nằm chung package `com.example.smartalarm` cho đơn giản.

Bước 1. Tạo project và cấu hình Gradle

1. Android Studio → **New Project** → **Empty Views Activity**.
2. Name: `SmartAlarm`; Package name: `com.example.smartalarm`; Language: **Java**; Minimum SDK: **API 26 (Android 8.0)**; Build configuration language: Kotlin DSL.
3. Mở `app/build.gradle.kts`, đặt `targetSdk = 34` và bổ sung thư viện RecyclerView, CoordinatorLayout, Core, rồi bấm **Sync Now**:

```
android {
    namespace = "com.example.smartalarm"
    compileSdk = 35

    defaultConfig {
        applicationId = "com.example.smartalarm"
        minSdk = 26
        targetSdk = 34   // giữ 34 để không bị ép chế độ edge-to-edge của
Android 15
        versionCode = 1
        versionName = "1.0"
    }
}

dependencies {
    implementation("androidx.appcompat:appcompat:1.7.0")
    implementation("com.google.android.material:material:1.12.0")
    implementation("androidx.recyclerview:recyclerview:1.3.2")
    implementation("androidx.coordinatorlayout:coordinatorlayout:1.2.0")
    implementation("androidx.core:core:1.13.1")
    implementation("androidx.activity:activity:1.9.3")
}
```

4. Xóa file `res/values-night/themes.xml` (ứng dụng tự quản lý màu sáng/tối).

Bước 2. Tài nguyên: màu, theme, style, drawable

Tạo/ghi đè các file sau trong thư mục `res`. Biểu tượng có thể tạo bằng **File → New → Vector Asset** (clip art “alarm” và “add”) thay vì gõ tay.

`res/values/colors.xml`

```
<resources>
    <color name="accent">#FF6B35</color>
    <color name="bg_screen">#F4F6FB</color>
    <color name="bg_card">#FFFFFF</color>
    <color name="text_primary">#1B1F2A</color>
    <color name="text_secondary">#6B7280</color>
    <color name="day_off">#E5E7EB</color>
    <color name="ring_bg">#14182B</color>
    <color name="ring_button">#2A3150</color>
    <color name="ring_text_secondary">#AAB2D0</color>
</resources>
```

`res/values/themes.xml`

```
<resources>
    <!-- Theme chung của ứng dụng (có ActionBar để hiện tiêu đề màn hình) -->
    <style name="Theme.SmartAlarm" parent="Theme.Material3.Light">
        <item name="colorPrimary">@color/accent</item>
        <item name="colorSecondary">@color/accent</item>
        <item name="android:statusBarColor">@color/bg_screen</item>
        <item name="android:windowLightStatusBar">true</item>
    </style>

    <!-- Theme tối, không ActionBar cho màn hình báo thức đang kêu -->
    <style name="Theme.SmartAlarm.Ring"
parent="Theme.Material3.Dark.NoActionBar">
        <item name="colorPrimary">@color/accent</item>
        <item name="android:statusBarColor">@color/ring_bg</item>
        <item name="android:navigationBarColor">@color/ring_bg</item>
    </style>
</resources>
```

`res/values/styles.xml`

```
<resources>
    <!-- Tiêu đề từng nhóm trên màn hình Thêm/Sửa -->
    <style name="SectionTitle">
        <item name="android:layout_width">match_parent</item>
        <item name="android:layout_height">wrap_content</item>
        <item name="android:layout_marginTop">20dp</item>
        <item name="android:layout_marginBottom">4dp</item>
```

```
        <item name="android:textColor">@color/accent</item>
        <item name="android:textSize">14sp</item>
        <item name="android:textStyle">bold</item>
    </style>

    <!-- Ô tròn chọn ngày T2..CN -->
    <style name="DayCircle">
        <item name="android:layout_width">38dp</item>
        <item name="android:layout_height">38dp</item>
        <item name="android:layout_marginEnd">5dp</item>
        <item name="android:button">@null</item>
        <item name="android:gravity">center</item>
        <item name="android:background">@drawable/bg_day_selector</item>
        <item name="android:textColor">@color/day_text_selector</item>
        <item name="android:textSize">13sp</item>
        <item name="android:textStyle">bold</item>
    </style>

    <!-- Nút đáp án mini game Giải toán -->
    <style name="AnswerButton">
        <item name="android:layout_width">0dp</item>
        <item name="android:layout_height">72dp</item>
        <item name="android:layout_weight">1</item>
        <item name="android:layout_margin">6dp</item>
        <item name="android:textSize">24sp</item>
        <item name="android:textColor">@android:color/white</item>
        <item name="android:backgroundTint">@color/ring_button</item>
    </style>

    <!-- Ô số mini game Bấm theo thứ tự -->
    <style name="SequenceButton">
        <item name="android:layout_width">88dp</item>
        <item name="android:layout_height">88dp</item>
        <item name="android:layout_margin">6dp</item>
        <item name="android:textSize">28sp</item>
        <item name="android:textColor">@android:color/white</item>
        <item name="android:backgroundTint">@color/ring_button</item>
    </style>
</resources>
```

`res/color/day_text_selector.xml` (tạo thư mục `color` nếu chưa có)

```
<selector xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:color="@android:color/white" android:state_checked="true"
/>
    <item android:color="@color/text_secondary" />
</selector>
```

`res/drawable/bg_day_selector.xml`

```
<selector xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:state_checked="true">
        <shape android:shape="oval"><solid android:color="@color/accent" />
</shape>
    </item>
    <item>
        <shape android:shape="oval"><solid android:color="@color/day_off" />
</shape>
    </item>
</selector>
```

`res/drawable/bg_pill.xml`

```
<shape xmlns:android="http://schemas.android.com/apk/res/android"
android:shape="rectangle">
    <solid android:color="@color/accent" />
    <corners android:radius="999dp" />
</shape>
```

`res/drawable/ic_alarm.xml` và `res/drawable/ic_add.xml`

```
<!-- ic_alarm.xml -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp" android:height="24dp"
    android:viewportWidth="24" android:viewportHeight="24">
    <path android:fillColor="#FFFFFFFF"
        android:pathData="M22,5.72l-4.6,-3.86 -1.29,1.53
4.6,3.86L22,5.72zM7.88,3.39L6.6,1.86 2,5.71l1.29,1.53
4.59,-3.85zM12.5,8L11,8v6l4.75,2.85 0.75,-1.23 -4,-2.37L12.5,8zM12,4c-4.97,0
-9,4.03 -9,9s4.02,9 9,9c4.97,0 9,-4.03 9,-9s-4.03,-9 -9,-9zM12,20c-3.87,0
-7,-3.13 -7,-7s3.13,-7 7,-7 7,3.13 7,7 -3.13,7 -7,7z" />
</vector>

<!-- ic_add.xml -->
<vector xmlns:android="http://schemas.android.com/apk/res/android"
    android:width="24dp" android:height="24dp"
    android:viewportWidth="24" android:viewportHeight="24">
    <path android:fillColor="#FFFFFFFF" android:pathData="M19,13h-6v6h-2v-
6H5v-2h6V5h2v6h6v2z" />
</vector>
```

Bước 3. Thiết kế layout cho từng màn hình

Năm file layout tương ứng với mockup ở phần 3. Chữ tiếng Việt viết trực tiếp trong XML cho dễ đọc; khi làm sản phẩm thật nên chuyển vào `strings.xml`.

Màn hình ① – `res/layout/activity_main.xml` (ghi đè file mặc định)

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/bg_screen">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical">

        <!-- Đồng hồ hiện tại -->
        <TextClock
```

```
            android:id="@+id/tcNow"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:format24Hour="HH:mm"
            android:format12Hour="HH:mm"
            android:gravity="center"
            android:paddingTop="24dp"
            android:textColor="@color/text_primary"
            android:textSize="56sp"
            android:textStyle="bold" />

        <TextClock
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:format24Hour="EEEE, dd/MM/yyyy"
            android:format12Hour="EEEE, dd/MM/yyyy"
            android:gravity="center"
            android:paddingBottom="16dp"
            android:textColor="@color/text_secondary"
            android:textSize="16sp" />

        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/rvAlarms"
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"
            android:clipToPadding="false"
            android:paddingStart="16dp"
            android:paddingEnd="16dp"
            android:paddingBottom="96dp" />
    </LinearLayout>

    <!-- Hiện khi chưa có báo thức nào -->
    <TextView
        android:id="@+id/tvEmpty"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="center"
        android:gravity="center"
        android:text="Chưa có báo thức nào.\nBấm + để thêm báo thức."
        android:textColor="@color/text_secondary"
        android:textSize="16sp"
        android:visibility="gone" />
```

```
    <com.google.android.material.floatingactionbutton.FloatingActionButton
        android:id="@+id/fabAdd"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_gravity="bottom|center_horizontal"
        android:layout_margin="24dp"
        android:contentDescription="Thêm báo thức"
        app:backgroundTint="@color/accent"
        app:srcCompat="@drawable/ic_add"
        app:tint="@android:color/white" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>
```

Một dòng trong danh sách – `res/layout/item_alarm.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_marginTop="6dp"
    android:layout_marginBottom="6dp"
    app:cardBackgroundColor="@color/bg_card"
    app:cardCornerRadius="16dp"
    app:cardElevation="0dp"
    app:strokeWidth="0dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:gravity="center_vertical"
        android:orientation="horizontal"
        android:padding="16dp">

        <LinearLayout
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:orientation="vertical">

            <TextView
```

```
                android:id="@+id/tvTime"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="06:30"
                android:textColor="@color/text_primary"
                android:textSize="36sp"
                android:textStyle="bold" />

            <TextView
                android:id="@+id/tvLabel"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Dậy đi học"
                android:textColor="@color/text_primary"
                android:textSize="15sp" />

            <TextView
                android:id="@+id/tvRepeat"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="T2, T3, T4, T5, T6"
                android:textColor="@color/accent"
                android:textSize="13sp" />

            <TextView
                android:id="@+id/tvExtra"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_marginTop="2dp"
                android:text="♪ Mặc định  •  Giải 3 phép toán"
                android:textColor="@color/text_secondary"
                android:textSize="12sp" />
        </LinearLayout>

        <androidx.appcompat.widget.SwitchCompat
            android:id="@+id/swEnabled"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content" />
    </LinearLayout>
</com.google.android.material.card.MaterialCardView>
```

Màn hình ② – `res/layout/activity_add_edit_alarm.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/bg_screen"
    android:orientation="vertical">

    <ScrollView
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="vertical"
            android:padding="16dp">

            <!-- 1. Chọn giờ -->
            <TimePicker
                android:id="@+id/timePicker"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:layout_gravity="center_horizontal"
                android:timePickerMode="spinner" />

            <!-- 2. Nhãn -->
            <TextView style="@style/SectionTitle" android:text="Nhãn báo
thức" />

            <EditText
                android:id="@+id/edtLabel"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:hint="Ví dụ: Dậy đi học"
                android:inputType="textCapSentences"
                android:maxLength="40" />

            <!-- 3. Kiểu lặp -->
            <TextView style="@style/SectionTitle" android:text="Lặp lại" />

            <RadioGroup
                android:id="@+id/rgRepeat"
```

```
            android:layout_width="match_parent"
            android:layout_height="wrap_content">

            <RadioButton
                android:id="@+id/rbOnce"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Một lần" />

            <RadioButton
                android:id="@+id/rbDaily"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Hằng ngày" />

            <RadioButton
                android:id="@+id/rbWeekly"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Hằng tuần (một thứ cố định)" />

            <RadioButton
                android:id="@+id/rbCustom"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Tùy chọn các ngày trong tuần" />
        </RadioGroup>

        <!-- 3a. Hằng tuần: chọn 1 thứ -->
        <LinearLayout
            android:id="@+id/layoutWeekly"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center_vertical"
            android:orientation="horizontal"
            android:paddingStart="32dp"
            android:visibility="gone">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Vào: "
                android:textColor="@color/text_primary" />
```

```
                <Spinner
                    android:id="@+id/spWeekday"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content" />
            </LinearLayout>

            <!-- 3b. Tùy chọn: 7 ô tròn T2..CN -->
            <LinearLayout
                android:id="@+id/layoutDays"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="8dp"
                android:orientation="horizontal"
                android:visibility="gone">

                <CheckBox android:id="@+id/cbMon" style="@style/DayCircle"
android:text="T2" />
                <CheckBox android:id="@+id/cbTue" style="@style/DayCircle"
android:text="T3" />
                <CheckBox android:id="@+id/cbWed" style="@style/DayCircle"
android:text="T4" />
                <CheckBox android:id="@+id/cbThu" style="@style/DayCircle"
android:text="T5" />
                <CheckBox android:id="@+id/cbFri" style="@style/DayCircle"
android:text="T6" />
                <CheckBox android:id="@+id/cbSat" style="@style/DayCircle"
android:text="T7" />
                <CheckBox android:id="@+id/cbSun" style="@style/DayCircle"
android:text="CN" />
            </LinearLayout>

            <!-- 4. Chuông & rung -->
            <TextView style="@style/SectionTitle" android:text="Âm thanh" />

            <LinearLayout
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:gravity="center_vertical"
                android:orientation="horizontal">

                <TextView
                    android:id="@+id/tvRingtone"
```

```
                    android:layout_width="0dp"
                    android:layout_height="wrap_content"
                    android:layout_weight="1"
                    android:text="Mặc định"
                    android:textColor="@color/text_primary"
                    android:textSize="16sp" />

                <Button
                    android:id="@+id/btnPickRingtone"
                    style="@style/Widget.Material3.Button.OutlinedButton"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:text="Chọn chuông" />
            </LinearLayout>

            <androidx.appcompat.widget.SwitchCompat
                android:id="@+id/swVibrate"
                android:layout_width="match_parent"
                android:layout_height="wrap_content"
                android:layout_marginTop="4dp"
                android:checked="true"
                android:text="Rung khi báo thức"
                android:textColor="@color/text_primary"
                android:textSize="16sp" />

            <!-- 5. Mini game -->
            <TextView style="@style/SectionTitle" android:text="Mini game để tắt báo thức" />

            <RadioGroup
                android:id="@+id/rgGame"
                android:layout_width="match_parent"
                android:layout_height="wrap_content">

                <RadioButton
                    android:id="@+id/rbGameMath"
                    android:layout_width="wrap_content"
                    android:layout_height="wrap_content"
                    android:checked="true"
                    android:text="Giải toán (chọn đáp án đúng)" />

                <RadioButton
                    android:id="@+id/rbGameSequence"
```

```
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Bấm các số 1 → 9 theo thứ tự" />
        </RadioGroup>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:gravity="center_vertical"
            android:orientation="horizontal">

            <TextView
                android:layout_width="wrap_content"
                android:layout_height="wrap_content"
                android:text="Số câu / vòng phải hoàn thành: "
                android:textColor="@color/text_primary" />

            <Spinner
                android:id="@+id/spRounds"
                android:layout_width="wrap_content"
                android:layout_height="wrap_content" />
        </LinearLayout>
    </LinearLayout>
</ScrollView>

<!-- Thanh nút cố định ở đáy -->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:orientation="horizontal"
    android:padding="16dp">

    <Button
        android:id="@+id/btnCancel"
        style="@style/Widget.Material3.Button.TonalButton"
        android:layout_width="0dp"
        android:layout_height="56dp"
        android:layout_marginEnd="8dp"
        android:layout_weight="1"
        android:text="Hủy" />

    <Button
        android:id="@+id/btnSave"
```

```
            android:layout_width="0dp"
            android:layout_height="56dp"
            android:layout_marginStart="8dp"
            android:layout_weight="1"
            android:backgroundTint="@color/accent"
            android:text="Lưu" />
    </LinearLayout>
</LinearLayout>
```

Màn hình ③ – `res/layout/activity_alarm_ring.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/ring_bg"
    android:gravity="center_horizontal"
    android:orientation="vertical"
    android:padding="24dp">

    <!-- Giờ hiện tại, tự cập nhật -->
    <TextClock
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="32dp"
        android:format12Hour="HH:mm"
        android:format24Hour="HH:mm"
        android:textColor="@android:color/white"
        android:textSize="72sp"
        android:textStyle="bold" />

    <TextView
        android:id="@+id/tvRingLabel"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Dậy đi học"
        android:textColor="@color/ring_text_secondary"
        android:textSize="20sp" />

    <TextView
        android:id="@+id/tvProgress"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```
        android:layout_marginTop="32dp"
        android:background="@drawable/bg_pill"
        android:paddingStart="16dp"
        android:paddingTop="6dp"
        android:paddingEnd="16dp"
        android:paddingBottom="6dp"
        android:text="Câu 1/3"
        android:textColor="@android:color/white"
        android:textSize="14sp" />

    <TextView
        android:id="@+id/tvQuestion"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="16dp"
        android:layout_marginBottom="24dp"
        android:gravity="center"
        android:text="47 + 38 = ?"
        android:textColor="@android:color/white"
        android:textSize="36sp"
        android:textStyle="bold" />

    <!-- Mini game 1: 4 đáp án dạng lưới 2x2 -->
    <LinearLayout
        android:id="@+id/panelMath"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical">

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal">

            <Button android:id="@+id/btnAnswer1" style="@style/AnswerButton"
/>
            <Button android:id="@+id/btnAnswer2" style="@style/AnswerButton"
/>
        </LinearLayout>

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
```

```
            android:orientation="horizontal">

            <Button android:id="@+id/btnAnswer3" style="@style/AnswerButton"
/>
            <Button android:id="@+id/btnAnswer4" style="@style/AnswerButton"
/>
        </LinearLayout>
    </LinearLayout>

    <!-- Mini game 2: lưới 3x3 số xáo trộn -->
    <LinearLayout
        android:id="@+id/panelSequence"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:visibility="gone">

        <GridLayout
            android:id="@+id/gridSequence"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:columnCount="3"
            android:rowCount="3">

            <Button style="@style/SequenceButton" />
            <Button style="@style/SequenceButton" />
            <Button style="@style/SequenceButton" />
            <Button style="@style/SequenceButton" />
            <Button style="@style/SequenceButton" />
            <Button style="@style/SequenceButton" />
            <Button style="@style/SequenceButton" />
            <Button style="@style/SequenceButton" />
            <Button style="@style/SequenceButton" />
        </GridLayout>
    </LinearLayout>

    <View
        android:layout_width="0dp"
        android:layout_height="0dp"
        android:layout_weight="1" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```
        android:text="Hoàn thành mini game để tắt chuông"
        android:textColor="@color/ring_text_secondary"
        android:textSize="13sp" />
</LinearLayout>
```

Notification ④ – `res/layout/notification_alarm.xml`. RemoteViews chỉ hỗ trợ một số View cơ bản (LinearLayout, TextView, ImageView…), không dùng được ConstraintLayout hay View tự viết.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="center_vertical"
    android:orientation="horizontal">

    <TextView
        android:id="@+id/tvNotifTime"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginEnd="12dp"
        android:text="06:30"
        android:textColor="#FF6B35"
        android:textSize="28sp"
        android:textStyle="bold" />

    <LinearLayout
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:orientation="vertical">

        <TextView
            android:id="@+id/tvNotifLabel"
            style="@style/TextAppearance.Compat.Notification.Title"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Dậy đi học" />

        <TextView
            android:id="@+id/tvNotifHint"
            style="@style/TextAppearance.Compat.Notification"
            android:layout_width="wrap_content"
```

```
            android:layout_height="wrap_content"
            android:text="Giải 3 phép toán để tắt chuông" />
    </LinearLayout>
</LinearLayout>
```

Bước 4. Lớp model `Alarm` và lưu trữ `AlarmStore`

`Alarm` giữ dữ liệu và có hàm quan trọng nhất của bài: `getNextTriggerTime()` – tìm thời điểm kêu kế tiếp cho cả 4 kiểu lặp. Chuột phải vào package → **New → Java Class** → `Alarm`.

```
package com.example.smartalarm;

import java.util.Calendar;
import java.util.Locale;

/** Lớp mô hình (model) mô tả một báo thức. */
public class Alarm {

    // Kiểu lặp lại
    public static final int REPEAT_ONCE = 0;    // Một lần
    public static final int REPEAT_DAILY = 1;   // Hằng ngày
    public static final int REPEAT_WEEKLY = 2;  // Hằng tuần (1 thứ cố định)
    public static final int REPEAT_CUSTOM = 3;  // Tùy chọn các ngày trong
tuần

    // Loại mini game
    public static final int GAME_MATH = 0;      // Giải toán
    public static final int GAME_SEQUENCE = 1;  // Bấm theo thứ tự 1 -> 9

    // Tên ngày, chỉ số 0 = Thứ Hai ... 6 = Chủ Nhật
    public static final String[] DAY_SHORT = {"T2", "T3", "T4", "T5", "T6",
"T7", "CN"};
    public static final String[] DAY_FULL = {"Thứ Hai", "Thứ Ba", "Thứ Tư",
            "Thứ Năm", "Thứ Sáu", "Thứ Bảy", "Chủ Nhật"};

    public int id;
    public int hour;
    public int minute;
    public String label = "";
    public boolean enabled = true;
    public int repeatType = REPEAT_ONCE;
    public int daysMask = 0;           // bit i = 1 nếu báo vào ngày i (0 =
```

```
T2)
    public String ringtoneUri = "";    // rỗng = chuông mặc định của hệ thống
    public String ringtoneTitle = "Mặc định";
    public boolean vibrate = true;
    public int gameType = GAME_MATH;
    public int gameRounds = 3;         // số câu/vòng phải hoàn thành

    /** Đổi Calendar.DAY_OF_WEEK (CN=1, T2=2 ... T7=7) sang chỉ số 0..6
(T2..CN). */
    public static int dayIndexOf(int calendarDayOfWeek) {
        return (calendarDayOfWeek + 5) % 7;
    }

    /** Mặt nạ ngày thực sự dùng để lập lịch, tùy theo kiểu lặp. */
    public int getEffectiveMask() {
        switch (repeatType) {
            case REPEAT_DAILY:  return 0x7F;          // cả 7 ngày
            case REPEAT_WEEKLY:
            case REPEAT_CUSTOM: return daysMask;
            default:            return 0;             // một lần
        }
    }

    /** Tính thời điểm (ms) báo thức kêu lần kế tiếp, tính từ thời điểm now.
*/
    public long getNextTriggerTime(long now) {
        Calendar c = Calendar.getInstance();
        c.setTimeInMillis(now);
        c.set(Calendar.HOUR_OF_DAY, hour);
        c.set(Calendar.MINUTE, minute);
        c.set(Calendar.SECOND, 0);
        c.set(Calendar.MILLISECOND, 0);

        int mask = getEffectiveMask();
        if (mask == 0) {                       // báo một lần
            if (c.getTimeInMillis() <= now) c.add(Calendar.DAY_OF_YEAR, 1);
            return c.getTimeInMillis();
        }
        // Lặp: tìm ngày gần nhất (tối đa 8 ngày) có bit được bật
        for (int i = 0; i < 8; i++) {
            int idx = dayIndexOf(c.get(Calendar.DAY_OF_WEEK));
            if ((mask & (1 << idx)) != 0 && c.getTimeInMillis() > now) {
                return c.getTimeInMillis();
```

```
            }
            c.add(Calendar.DAY_OF_YEAR, 1);
        }
        return c.getTimeInMillis();
    }

    public String getTimeText() {
        return String.format(Locale.getDefault(), "%02d:%02d", hour, minute);
    }

    /** Chuỗi mô tả kiểu lặp hiển thị trên danh sách. */
    public String getRepeatText() {
        switch (repeatType) {
            case REPEAT_DAILY:
                return "Hằng ngày";
            case REPEAT_WEEKLY:
                for (int i = 0; i < 7; i++) {
                    if ((daysMask & (1 << i)) != 0) return "Hằng tuần vào " +
DAY_FULL[i];
                }
                return "Hằng tuần";
            case REPEAT_CUSTOM:
                if (daysMask == 0x1F) return "Các ngày trong tuần (T2–T6)";
                if (daysMask == 0x60) return "Cuối tuần (T7, CN)";
                StringBuilder sb = new StringBuilder();
                for (int i = 0; i < 7; i++) {
                    if ((daysMask & (1 << i)) != 0) {
                        if (sb.length() > 0) sb.append(", ");
                        sb.append(DAY_SHORT[i]);
                    }
                }
                return sb.toString();
            default:
                return "Một lần";
        }
    }

    public String getGameText() {
        return gameType == GAME_MATH
                ? "Giải " + gameRounds + " phép toán"
                : "Bấm 1→9 theo thứ tự (" + gameRounds + " vòng)";
    }
}
```

Ví dụ kiểm tra (lúc 09:00 Thứ Ba 13/10/2026, báo thức 08:30): Một lần/Hăng ngày → 08:30 Thứ Tư 14/10; Hăng tuần vào Thứ Hai → 08:30 Thứ Hai 19/10; Cuối tuần → 08:30 Thứ Bảy 17/10.

`AlarmStore` lưu cả danh sách thành một chuỗi JSON trong SharedPreferences, tự cấp `id` tăng dần:

```
package com.example.smartalarm;

import android.content.Context;
import android.content.SharedPreferences;

import org.json.JSONArray;
import org.json.JSONException;
import org.json.JSONObject;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** Lưu danh sách báo thức vào SharedPreferences dưới dạng chuỗi JSON. */
public class AlarmStore {

    private static final String PREF = "smart_alarm_prefs";
    private static final String KEY_LIST = "alarms";
    private static final String KEY_NEXT_ID = "next_id";

    private static SharedPreferences prefs(Context ctx) {
        return ctx.getApplicationContext().getSharedPreferences(PREF,
Context.MODE_PRIVATE);
    }

    public static synchronized List<Alarm> getAll(Context ctx) {
        List<Alarm> list = new ArrayList<>();
        try {
            JSONArray arr = new JSONArray(prefs(ctx).getString(KEY_LIST, "
[]"));
            for (int i = 0; i < arr.length(); i++)
list.add(fromJson(arr.getJSONObject(i)));
        } catch (JSONException e) {
            e.printStackTrace();
        }
        // Săp xếp theo giờ:phút tăng dần
```

```
        Collections.sort(list, (a, b) -> (a.hour * 60 + a.minute) - (b.hour *
60 + b.minute));
        return list;
    }

    public static Alarm get(Context ctx, int id) {
        for (Alarm a : getAll(ctx)) if (a.id == id) return a;
        return null;
    }

    /** Thêm mới (id = 0) hoặc cập nhật báo thức đã có. */
    public static synchronized void save(Context ctx, Alarm alarm) {
        List<Alarm> list = getAll(ctx);
        if (alarm.id == 0) {
            int next = prefs(ctx).getInt(KEY_NEXT_ID, 1);
            alarm.id = next;
            prefs(ctx).edit().putInt(KEY_NEXT_ID, next + 1).apply();
            list.add(alarm);
        } else {
            boolean found = false;
            for (int i = 0; i < list.size(); i++) {
                if (list.get(i).id == alarm.id) { list.set(i, alarm); found =
true; break; }
            }
            if (!found) list.add(alarm);
        }
        writeAll(ctx, list);
    }

    public static synchronized void delete(Context ctx, int id) {
        List<Alarm> list = getAll(ctx);
        for (int i = list.size() - 1; i >= 0; i--) if (list.get(i).id == id)
list.remove(i);
        writeAll(ctx, list);
    }

    private static void writeAll(Context ctx, List<Alarm> list) {
        JSONArray arr = new JSONArray();
        try {
            for (Alarm a : list) arr.put(toJson(a));
        } catch (JSONException e) {
            e.printStackTrace();
        }
```

```
        prefs(ctx).edit().putString(KEY_LIST, arr.toString()).apply();
    }

    private static JSONObject toJson(Alarm a) throws JSONException {
        JSONObject o = new JSONObject();
        o.put("id", a.id);
        o.put("hour", a.hour);
        o.put("minute", a.minute);
        o.put("label", a.label);
        o.put("enabled", a.enabled);
        o.put("repeatType", a.repeatType);
        o.put("daysMask", a.daysMask);
        o.put("ringtoneUri", a.ringtoneUri);
        o.put("ringtoneTitle", a.ringtoneTitle);
        o.put("vibrate", a.vibrate);
        o.put("gameType", a.gameType);
        o.put("gameRounds", a.gameRounds);
        return o;
    }

    private static Alarm fromJson(JSONObject o) {
        Alarm a = new Alarm();
        a.id = o.optInt("id");
        a.hour = o.optInt("hour");
        a.minute = o.optInt("minute");
        a.label = o.optString("label", "");
        a.enabled = o.optBoolean("enabled", true);
        a.repeatType = o.optInt("repeatType", Alarm.REPEAT_ONCE);
        a.daysMask = o.optInt("daysMask", 0);
        a.ringtoneUri = o.optString("ringtoneUri", "");
        a.ringtoneTitle = o.optString("ringtoneTitle", "Mặc định");
        a.vibrate = o.optBoolean("vibrate", true);
        a.gameType = o.optInt("gameType", Alarm.GAME_MATH);
        a.gameRounds = o.optInt("gameRounds", 3);
        return a;
    }
}
```

Bước 5. Đặt lịch với `AlarmManager` – lớp `AlarmScheduler`

`AlarmManager` là dịch vụ hệ thống; đến giờ nó gửi `PendingIntent` tới `AlarmReceiver` kể cả khi ứng dụng đã đóng. `setAlarmClock()` kêu chính xác, đánh thức máy cả khi ở chế độ Doze và hiện biểu tượng đồng hồ trên thanh trạng thái. Mỗi báo thức dùng `requestCode = id` để có PendingIntent riêng, nhờ đó hủy đúng báo thức.

```
package com.example.smartalarm;

import android.app.AlarmManager;
import android.app.PendingIntent;
import android.content.Context;
import android.content.Intent;
import android.os.Build;

/** Đặt / hủy báo thức với AlarmManager. Khi đến giờ, hệ thống gửi Intent tới
AlarmReceiver. */
public class AlarmScheduler {

    public static final String ACTION_ALARM_FIRE =
"com.example.smartalarm.ACTION_ALARM_FIRE";
    public static final String EXTRA_ALARM_ID = "alarm_id";

    private static PendingIntent buildFireIntent(Context ctx, int alarmId) {
        Intent intent = new Intent(ctx, AlarmReceiver.class);   // Intent
tường minh
        intent.setAction(ACTION_ALARM_FIRE);
        intent.putExtra(EXTRA_ALARM_ID, alarmId);
        // requestCode = alarmId để mỗi báo thức có một PendingIntent riêng
        return PendingIntent.getBroadcast(ctx, alarmId, intent,
                PendingIntent.FLAG_UPDATE_CURRENT |
PendingIntent.FLAG_IMMUTABLE);
    }

    /** Đặt báo thức cho lần kêu kế tiếp. Trả về thời điểm (ms) sẽ kêu. */
    public static long schedule(Context ctx, Alarm alarm) {
        AlarmManager am = ctx.getSystemService(AlarmManager.class);
        long triggerAt =
alarm.getNextTriggerTime(System.currentTimeMillis());
        PendingIntent operation = buildFireIntent(ctx, alarm.id);

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S &&
!am.canScheduleExactAlarms()) {
```

```
            // Chưa được cấp quyền báo thức chính xác: dùng phương án gần
đúng
            am.setAndAllowWhileIdle(AlarmManager.RTC_WAKEUP, triggerAt,
operation);
            return triggerAt;
        }
        // setAlarmClock: chính xác, đánh thức máy cả khi Doze, hiện biểu
tượng đồng hồ trên status bar
        Intent showIntent = new Intent(ctx, MainActivity.class);
        PendingIntent show = PendingIntent.getActivity(ctx, alarm.id,
showIntent,
                PendingIntent.FLAG_UPDATE_CURRENT |
PendingIntent.FLAG_IMMUTABLE);
        am.setAlarmClock(new AlarmManager.AlarmClockInfo(triggerAt, show),
operation);
        return triggerAt;
    }

    public static void cancel(Context ctx, int alarmId) {
        AlarmManager am = ctx.getSystemService(AlarmManager.class);
        am.cancel(buildFireIntent(ctx, alarmId));
    }

    /** Đặt lại toàn bộ báo thức đang bật (dùng sau khi khởi động lại máy).
*/
    public static void scheduleAll(Context ctx) {
        for (Alarm a : AlarmStore.getAll(ctx)) {
            if (a.enabled) schedule(ctx, a);
        }
    }

    /** Chuỗi "Báo thức sẽ kêu sau X giờ Y phút" để hiện Toast. */
    public static String timeUntilText(long triggerAt) {
        long diffMin = Math.max(0, (triggerAt - System.currentTimeMillis() +
59_999) / 60_000);
        long days = diffMin / (24 * 60);
        long hours = (diffMin % (24 * 60)) / 60;
        long mins = diffMin % 60;
        StringBuilder sb = new StringBuilder("Báo thức sẽ kêu sau ");
        if (days > 0) sb.append(days).append(" ngày ");
        if (hours > 0) sb.append(hours).append(" giờ ");
```

```
        sb.append(mins).append(" phút");
        return sb.toString();
    }
}
```

Bước 6. Broadcast Receiver: `AlarmReceiver` và `BootReceiver`

Cả hai receiver đều **khai báo tĩnh** trong Manifest (Bước 11), nên hệ thống gọi được `onReceive()` ngay cả khi ứng dụng chưa chạy. `AlarmReceiver` làm 3 việc: đặt lịch lần sau, khởi động Foreground Service, gửi broadcast nội bộ có permission.

`AlarmReceiver.java`

```
package com.example.smartalarm;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;

import androidx.core.content.ContextCompat;

/**
 * Broadcast Receiver khai báo TĨNH trong AndroidManifest.
 * AlarmManager gửi Intent tới đây khi đến giờ báo thức.
 */
public class AlarmReceiver extends BroadcastReceiver {

    /** Broadcast nội bộ: báo cho MainActivity biết danh sách vừa thay đổi.
*/
    public static final String ACTION_ALARMS_CHANGED =
"com.example.smartalarm.ALARMS_CHANGED";
    /** Permission tự định nghĩa (protectionLevel = signature) bảo vệ
broadcast nội bộ. */
    public static final String PERMISSION_INTERNAL =
"com.example.smartalarm.permission.INTERNAL_BROADCAST";

    @Override
    public void onReceive(Context context, Intent intent) {
        if (!AlarmScheduler.ACTION_ALARM_FIRE.equals(intent.getAction()))
return;

        int alarmId = intent.getIntExtra(AlarmScheduler.EXTRA_ALARM_ID, -1);
        Alarm alarm = AlarmStore.get(context, alarmId);
```

```
        if (alarm == null || !alarm.enabled) return;

        // 1. Báo một lần -> tắt; báo lặp -> đặt lịch cho lần kế tiếp
        if (alarm.repeatType == Alarm.REPEAT_ONCE) {
            alarm.enabled = false;
            AlarmStore.save(context, alarm);
        } else {
            AlarmScheduler.schedule(context, alarm);
        }

        // 2. Khởi động Foreground Service để phát chuông
        Intent service = new Intent(context, AlarmService.class);
        service.setAction(AlarmService.ACTION_START);
        service.putExtra(AlarmScheduler.EXTRA_ALARM_ID, alarmId);
        ContextCompat.startForegroundService(context, service);

        // 3. Gửi broadcast nội bộ (có permission) để màn hình danh sách tự
làm mới
        Intent changed = new Intent(ACTION_ALARMS_CHANGED);
        changed.setPackage(context.getPackageName());
        context.sendBroadcast(changed, PERMISSION_INTERNAL);
    }
}
```

`BootReceiver.java` – báo thức đặt trong `AlarmManager` bị xóa khi tắt máy, nên phải đặt lại khi nhận `BOOT_COMPLETED`:

```
package com.example.smartalarm;

import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;

/** Nhận broadcast hệ thống BOOT_COMPLETED để đặt lại báo thức (AlarmManager
bị xóa khi tắt máy). */
public class BootReceiver extends BroadcastReceiver {
    @Override
    public void onReceive(Context context, Intent intent) {
        if (Intent.ACTION_BOOT_COMPLETED.equals(intent.getAction())) {
            AlarmScheduler.scheduleAll(context);
        }
    }
}
```

Bước 7. `AlarmService` – Foreground Service + Bound Service + Notification

Đây là lớp trung tâm, gộp cả 3 nội dung của bài:

- **Foreground Service**: `startForeground()` phải được gọi trong vài giây sau `startForegroundService()`, nếu không ứng dụng bị lỗi ANR/crash. Android 14 bắt buộc khai báo loại `mediaPlayback`.
- **Bound Service**: lớp `LocalBinder` trả về chính service, client gọi trực tiếp `stopAlarm()`.
- **Notification**: custom layout bằng `RemoteViews`, nút action, `setFullScreenIntent()` để tự mở màn hình mini game khi máy đang khóa; `setOngoing(true)` để không vuốt bỏ được.
- **Vòng đời**: `onCreate()` tạo kênh thông báo; `onStartCommand()` bắt đầu kêu; `onBind()` trả IBinder; `onDestroy()` giải phóng `MediaPlayer`.

```
package com.example.smartalarm;

import android.app.Notification;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.app.PendingIntent;
import android.app.Service;
import android.content.Intent;
```

```
import android.content.pm.ServiceInfo;
import android.media.AudioAttributes;
import android.media.MediaPlayer;
import android.media.RingtoneManager;
import android.net.Uri;
import android.os.Binder;
import android.os.Build;
import android.os.IBinder;
import android.os.VibrationEffect;
import android.os.Vibrator;
import android.widget.RemoteViews;

import androidx.core.app.NotificationCompat;
import androidx.core.app.ServiceCompat;

/**
 * Foreground Service phát chuông + rung khi báo thức kêu.
 * Đồng thời là Bound Service: AlarmRingActivity bind vào để gọi stopAlarm().
 */
public class AlarmService extends Service {

    public static final String ACTION_START =
"com.example.smartalarm.ACTION_START";
    public static final String CHANNEL_ID = "alarm_channel";
    public static final int NOTIFICATION_ID = 1001;

    private MediaPlayer player;
    private Vibrator vibrator;
    private Alarm currentAlarm;
    private final IBinder binder = new LocalBinder();

    /** Binder cục bộ: trả về chính đối tượng Service cho client. */
    public class LocalBinder extends Binder {
        public AlarmService getService() {
            return AlarmService.this;
        }
    }

    // ---------------- Vòng đời Service ----------------

    @Override
    public void onCreate() {
        super.onCreate();
```

```
        vibrator = getSystemService(Vibrator.class);
        createNotificationChannel();
    }

    @Override
    public int onStartCommand(Intent intent, int flags, int startId) {
        if (intent == null || !ACTION_START.equals(intent.getAction())) {
            stopSelf();
            return START_NOT_STICKY;
        }
        int alarmId = intent.getIntExtra(AlarmScheduler.EXTRA_ALARM_ID, -1);
        currentAlarm = AlarmStore.get(this, alarmId);
        if (currentAlarm == null) currentAlarm = new Alarm();   // phòng
trường hợp đã bị xóa

        // Bắt buộc gọi startForeground() trong vòng vài giây sau
startForegroundService()
        Notification notification = buildNotification(currentAlarm);
        int type = Build.VERSION.SDK_INT >= Build.VERSION_CODES.Q
                ? ServiceInfo.FOREGROUND_SERVICE_TYPE_MEDIA_PLAYBACK : 0;
        ServiceCompat.startForeground(this, NOTIFICATION_ID, notification,
type);

        startRinging(currentAlarm);
        return START_NOT_STICKY;
    }

    @Override
    public IBinder onBind(Intent intent) {
        return binder;
    }

    @Override
    public void onDestroy() {
        stopRinging();
        super.onDestroy();
    }

    // ---------------- API cho client (Bound Service) ----------------

    public Alarm getCurrentAlarm() {
        return currentAlarm;
    }
```

```
    /** Gọi khi người dùng hoàn thành mini game. */
    public void stopAlarm() {
        stopRinging();
        ServiceCompat.stopForeground(this,
ServiceCompat.STOP_FOREGROUND_REMOVE);
        stopSelf();
    }

    // ---------------- Phát chuông & rung ----------------

    private void startRinging(Alarm alarm) {
        stopRinging();   // nếu đang kêu báo thức khác thì dừng trước

        Uri uri = alarm.ringtoneUri.isEmpty() ? null :
Uri.parse(alarm.ringtoneUri);
        if (uri == null) uri =
RingtoneManager.getDefaultUri(RingtoneManager.TYPE_ALARM);
        if (uri == null) uri =
RingtoneManager.getDefaultUri(RingtoneManager.TYPE_RINGTONE);

        try {
            player = new MediaPlayer();
            player.setAudioAttributes(new AudioAttributes.Builder()
                    .setUsage(AudioAttributes.USAGE_ALARM)            // dùng
luồng âm lượng báo thức

.setContentType(AudioAttributes.CONTENT_TYPE_SONIFICATION)
                    .build());
            player.setDataSource(this, uri);
            player.setLooping(true);                                  // kêu
liên tục
            player.prepare();
            player.start();
        } catch (Exception e) {
            e.printStackTrace();
            // Chuông đã chọn lỗi (bị xóa...) -> thử chuông mặc định
            if (!alarm.ringtoneUri.isEmpty()) {
                alarm.ringtoneUri = "";
                startRinging(alarm);
                return;
            }
        }
```

```
        if (alarm.vibrate && vibrator != null && vibrator.hasVibrator()) {
            long[] pattern = {0, 800, 600};   // chờ 0ms, rung 800ms, nghỉ
600ms
            vibrator.vibrate(VibrationEffect.createWaveform(pattern, 0),
                    new
AudioAttributes.Builder().setUsage(AudioAttributes.USAGE_ALARM).build());
        }
    }

    private void stopRinging() {
        if (player != null) {
            try { player.stop(); } catch (IllegalStateException ignored) { }
            player.release();
            player = null;
        }
        if (vibrator != null) vibrator.cancel();
    }

    // ---------------- Notification ----------------

    private void createNotificationChannel() {
        NotificationChannel channel = new NotificationChannel(CHANNEL_ID,
"Báo thức",
                NotificationManager.IMPORTANCE_HIGH);
        channel.setDescription("Thông báo khi báo thức đang kêu");
        channel.setSound(null, null);          // âm thanh do MediaPlayer
phát
        channel.enableVibration(false);        // rung do Vibrator xử lý
        channel.setLockscreenVisibility(Notification.VISIBILITY_PUBLIC);

getSystemService(NotificationManager.class).createNotificationChannel(channel
);
    }

    private Notification buildNotification(Alarm alarm) {
        // Intent mở màn hình mini game
        Intent ringIntent = new Intent(this, AlarmRingActivity.class);
        ringIntent.putExtra(AlarmScheduler.EXTRA_ALARM_ID, alarm.id);
        ringIntent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK |
Intent.FLAG_ACTIVITY_NO_USER_ACTION);
        PendingIntent ringPending = PendingIntent.getActivity(this, alarm.id,
ringIntent,
```

```
                PendingIntent.FLAG_UPDATE_CURRENT |
PendingIntent.FLAG_IMMUTABLE);

        // Custom Layout Notification (RemoteViews)
        RemoteViews rv = new RemoteViews(getPackageName(),
R.layout.notification_alarm);
        rv.setTextViewText(R.id.tvNotifTime, alarm.getTimeText());
        rv.setTextViewText(R.id.tvNotifLabel,
                alarm.label.isEmpty() ? "Báo thức" : alarm.label);
        rv.setTextViewText(R.id.tvNotifHint, alarm.getGameText() + " để tắt
chuông");

        return new NotificationCompat.Builder(this, CHANNEL_ID)
                .setSmallIcon(R.drawable.ic_alarm)
                .setContentTitle("Báo thức " + alarm.getTimeText())
                .setContentText(alarm.getGameText() + " để tắt chuông")
                .setCustomContentView(rv)
                .setStyle(new NotificationCompat.DecoratedCustomViewStyle())
                .setPriority(NotificationCompat.PRIORITY_MAX)
                .setCategory(NotificationCompat.CATEGORY_ALARM)
                .setVisibility(NotificationCompat.VISIBILITY_PUBLIC)
                .setOngoing(true)                               // không vuốt
bỏ được
                .setAutoCancel(false)
                .setContentIntent(ringPending)
                .setFullScreenIntent(ringPending, true)         // tự bật màn
hình mini game khi khóa máy
                .addAction(R.drawable.ic_alarm, "Giải mini game",
ringPending)   // Action Notification
                .build();
    }
}
```

Bước 8. Màn hình ①: `AlarmAdapter` và `MainActivity`

`AlarmAdapter` đổ dữ liệu lên `item_alarm.xml`. Chú ý gỡ listener trước khi `setChecked()`; nếu không, khi RecyclerView tái sử dụng View, công tắc sẽ gọi nhầm sự kiện của báo thức cũ.

```
package com.example.smartalarm;

import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;

import androidx.annotation.NonNull;
import androidx.appcompat.widget.SwitchCompat;
import androidx.recyclerview.widget.RecyclerView;

import java.util.ArrayList;
import java.util.List;

/** Adapter hiển thị danh sách báo thức trong RecyclerView. */
public class AlarmAdapter extends RecyclerView.Adapter<AlarmAdapter.Holder> {

    public interface Listener {
        void onClick(Alarm alarm);                      // chạm: sửa
        void onLongClick(Alarm alarm);                  // nhấn giữ: xóa
        void onToggle(Alarm alarm, boolean enabled);    // bật/tắt
    }

    private final List<Alarm> items = new ArrayList<>();
    private final Listener listener;

    public AlarmAdapter(Listener listener) {
        this.listener = listener;
    }

    public void submit(List<Alarm> list) {
        items.clear();
        items.addAll(list);
        notifyDataSetChanged();
    }

    @NonNull
    @Override
    public Holder onCreateViewHolder(@NonNull ViewGroup parent, int viewType)
{
        View v =
LayoutInflater.from(parent.getContext()).inflate(R.layout.item_alarm, parent,
false);
```

```
        return new Holder(v);
    }

    @Override
    public void onBindViewHolder(@NonNull Holder h, int position) {
        Alarm a = items.get(position);
        h.tvTime.setText(a.getTimeText());
        h.tvLabel.setText(a.label.isEmpty() ? "Báo thức" : a.label);
        h.tvRepeat.setText(a.getRepeatText());
        h.tvExtra.setText("♪ " + a.ringtoneTitle + "  •  " +
a.getGameText());

        // Gỡ listener trước khi setChecked để tránh gọi nhầm khi View được
tái sử dụng
        h.swEnabled.setOnCheckedChangeListener(null);
        h.swEnabled.setChecked(a.enabled);
        h.swEnabled.setOnCheckedChangeListener((btn, checked) ->
listener.onToggle(a, checked));

        float alpha = a.enabled ? 1f : 0.45f;
        h.tvTime.setAlpha(alpha);
        h.tvLabel.setAlpha(alpha);
        h.tvRepeat.setAlpha(alpha);

        h.itemView.setOnClickListener(v -> listener.onClick(a));
        h.itemView.setOnLongClickListener(v -> { listener.onLongClick(a);
return true; });
    }

    @Override
    public int getItemCount() {
        return items.size();
    }

    static class Holder extends RecyclerView.ViewHolder {
        final TextView tvTime, tvLabel, tvRepeat, tvExtra;
        final SwitchCompat swEnabled;

        Holder(View v) {
            super(v);
            tvTime = v.findViewById(R.id.tvTime);
            tvLabel = v.findViewById(R.id.tvLabel);
            tvRepeat = v.findViewById(R.id.tvRepeat);
```

```
            tvExtra = v.findViewById(R.id.tvExtra);
            swEnabled = v.findViewById(R.id.swEnabled);
        }
    }
}
```

`MainActivity` minh họa **receiver khai báo động**: đăng ký trong `onStart()`, gỡ trong `onStop()`, chỉ nhận broadcast có permission nội bộ. Nó cũng xin các quyền mới của Android 12/13/14. Ghi đè toàn bộ file mặc định:

```
package com.example.smartalarm;

import android.Manifest;
import android.app.AlarmManager;
import android.app.NotificationManager;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;
import android.content.pm.PackageManager;
import android.net.Uri;
import android.os.Build;
import android.os.Bundle;
import android.provider.Settings;
import android.view.View;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.result.ActivityResultLauncher;
import androidx.activity.result.contract.ActivityResultContracts;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;

import com.google.android.material.floatingactionbutton.FloatingActionButton;

import java.util.List;

/** Màn hình 1: danh sách báo thức. */
public class MainActivity extends AppCompatActivity implements
AlarmAdapter.Listener {
```

```
    private AlarmAdapter adapter;
    private TextView tvEmpty;

    // Receiver khai báo ĐỘNG: chỉ sống khi Activity đang hiển thị
    private final BroadcastReceiver alarmsChangedReceiver = new
BroadcastReceiver() {
        @Override
        public void onReceive(Context context, Intent intent) {
            loadAlarms();
        }
    };

    private final ActivityResultLauncher<String> notificationPermission =
            registerForActivityResult(new
ActivityResultContracts.RequestPermission(), granted -> {
                if (!granted) {
                    Toast.makeText(this, "Không có quyền thông báo, báo thức
có thể không hiện!",
                            Toast.LENGTH_LONG).show();
                }
            });

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        RecyclerView rv = findViewById(R.id.rvAlarms);
        tvEmpty = findViewById(R.id.tvEmpty);
        FloatingActionButton fab = findViewById(R.id.fabAdd);

        adapter = new AlarmAdapter(this);
        rv.setLayoutManager(new LinearLayoutManager(this));
        rv.setAdapter(adapter);

        fab.setOnClickListener(v -> startActivity(new Intent(this,
AddEditAlarmActivity.class)));

        checkPermissions();
    }

    @Override
```

```
    protected void onStart() {
        super.onStart();
        IntentFilter filter = new
IntentFilter(AlarmReceiver.ACTION_ALARMS_CHANGED);
        ContextCompat.registerReceiver(this, alarmsChangedReceiver, filter,
                AlarmReceiver.PERMISSION_INTERNAL, null,
ContextCompat.RECEIVER_NOT_EXPORTED);
        loadAlarms();
    }

    @Override
    protected void onStop() {
        unregisterReceiver(alarmsChangedReceiver);   // tránh rò rỉ bộ nhớ
        super.onStop();
    }

    private void loadAlarms() {
        List<Alarm> list = AlarmStore.getAll(this);
        adapter.submit(list);
        tvEmpty.setVisibility(list.isEmpty() ? View.VISIBLE : View.GONE);
    }

    // ---------------- Sự kiện từ Adapter ----------------

    @Override
    public void onClick(Alarm alarm) {
        Intent i = new Intent(this, AddEditAlarmActivity.class);
        i.putExtra(AlarmScheduler.EXTRA_ALARM_ID, alarm.id);
        startActivity(i);
    }

    @Override
    public void onLongClick(Alarm alarm) {
        new AlertDialog.Builder(this)
                .setTitle("Xóa báo thức")
                .setMessage("Xóa báo thức " + alarm.getTimeText() + "?")
                .setPositiveButton("Xóa", (d, w) -> {
                    AlarmScheduler.cancel(this, alarm.id);
                    AlarmStore.delete(this, alarm.id);
                    loadAlarms();
                })
                .setNegativeButton("Hủy", null)
                .show();
```

```
    }

    @Override
    public void onToggle(Alarm alarm, boolean enabled) {
        alarm.enabled = enabled;
        AlarmStore.save(this, alarm);
        if (enabled) {
            long at = AlarmScheduler.schedule(this, alarm);
            Toast.makeText(this, AlarmScheduler.timeUntilText(at),
Toast.LENGTH_SHORT).show();
        } else {
            AlarmScheduler.cancel(this, alarm.id);
        }
        loadAlarms();
    }

    // ---------------- Quyền (Android 12, 13, 14+) ----------------

    private void checkPermissions() {
        // Android 13+: quyền hiển thị thông báo
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU
                && ContextCompat.checkSelfPermission(this,
Manifest.permission.POST_NOTIFICATIONS)
                != PackageManager.PERMISSION_GRANTED) {

notificationPermission.launch(Manifest.permission.POST_NOTIFICATIONS);
        }
        // Android 12+: quyền đặt báo thức chính xác
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.S) {
            AlarmManager am = getSystemService(AlarmManager.class);
            if (!am.canScheduleExactAlarms()) {
                askOpenSettings("Cho phép đặt báo thức chính xác để báo thức
kêu đúng giờ.",
                        new
Intent(Settings.ACTION_REQUEST_SCHEDULE_EXACT_ALARM,
                                Uri.parse("package:" + getPackageName())));
            }
        }
        // Android 14+: quyền hiển thị toàn màn hình khi khóa máy
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.UPSIDE_DOWN_CAKE) {
            NotificationManager nm =
getSystemService(NotificationManager.class);
            if (!nm.canUseFullScreenIntent()) {
```

```
                askOpenSettings("Cho phép hiển thị toàn màn hình để mini game
hiện khi khóa máy.",
                        new
Intent(Settings.ACTION_MANAGE_APP_USE_FULL_SCREEN_INTENT,
                                Uri.parse("package:" + getPackageName())));
            }
        }
    }

    private void askOpenSettings(String message, Intent settingsIntent) {
        new AlertDialog.Builder(this)
                .setTitle("Cần cấp quyền")
                .setMessage(message)
                .setPositiveButton("Mở cài đặt", (d, w) ->
startActivity(settingsIntent))
                .setNegativeButton("Để sau", null)
                .show();
    }
}
```

Bước 9. Màn hình ②: `AddEditAlarmActivity` (chọn kiểu lặp, chọn chuông)

Tạo bằng **New → Activity → Empty Views Activity**, bỏ chọn "Generate Layout File" (đã có layout ở Bước 3). Điểm chính:

- `updateRepeatPanels()` chỉ hiện `Spinner` chọn thứ khi chọn **Hăng tuần**, chỉ hiện 7 ô T2–CN khi chọn **Tùy chọn**.
- `saveAlarm()` quy mọi kiểu lặp về `daysMask` (Bảng 2.2); kiểu Tùy chọn bắt buộc chọn ít nhất 1 ngày.
- `showRingtoneDialog()` dùng `RingtoneManager` lấy danh sách âm báo thức của hệ thống, chạm vào tên để nghe thử bằng `Ringtone.play()`.

```
package com.example.smartalarm;

import android.database.Cursor;
import android.media.Ringtone;
import android.media.RingtoneManager;
import android.net.Uri;
import android.os.Bundle;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.CheckBox;
```

```
import android.widget.EditText;
import android.widget.LinearLayout;
import android.widget.RadioGroup;
import android.widget.Spinner;
import android.widget.TextView;
import android.widget.TimePicker;
import android.widget.Toast;

import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.SwitchCompat;

import java.util.ArrayList;
import java.util.List;

/** Màn hình 2: thêm mới / sửa báo thức. */
public class AddEditAlarmActivity extends AppCompatActivity {

    private TimePicker timePicker;
    private EditText edtLabel;
    private RadioGroup rgRepeat, rgGame;
    private LinearLayout layoutWeekly, layoutDays;
    private Spinner spWeekday, spRounds;
    private final CheckBox[] dayBoxes = new CheckBox[7];
    private TextView tvRingtone;
    private SwitchCompat swVibrate;

    private Alarm alarm;                // báo thức đang sửa (hoặc mới)
    private Ringtone previewRingtone;   // chuông nghe thử

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_add_edit_alarm);

        bindViews();
        setupSpinners();

        int alarmId = getIntent().getIntExtra(AlarmScheduler.EXTRA_ALARM_ID,
0);
        Alarm existing = alarmId > 0 ? AlarmStore.get(this, alarmId) : null;
        if (existing != null) {
            alarm = existing;
```

```
            setTitle("Sửa báo thức");
        } else {
            alarm = new Alarm();
            java.util.Calendar now = java.util.Calendar.getInstance();
            alarm.hour = now.get(java.util.Calendar.HOUR_OF_DAY);
            alarm.minute = now.get(java.util.Calendar.MINUTE);
            alarm.daysMask = 0x1F;  // gợi ý sẵn T2–T6 cho chế độ tùy chọn
            setTitle("Thêm báo thức");
        }
        fillForm();

        rgRepeat.setOnCheckedChangeListener((group, checkedId) ->
updateRepeatPanels());
        findViewById(R.id.btnPickRingtone).setOnClickListener(v ->
showRingtoneDialog());
        findViewById(R.id.btnCancel).setOnClickListener(v -> finish());
        findViewById(R.id.btnSave).setOnClickListener(v -> saveAlarm());
    }

    private void bindViews() {
        timePicker = findViewById(R.id.timePicker);
        timePicker.setIs24HourView(true);
        edtLabel = findViewById(R.id.edtLabel);
        rgRepeat = findViewById(R.id.rgRepeat);
        rgGame = findViewById(R.id.rgGame);
        layoutWeekly = findViewById(R.id.layoutWeekly);
        layoutDays = findViewById(R.id.layoutDays);
        spWeekday = findViewById(R.id.spWeekday);
        spRounds = findViewById(R.id.spRounds);
        tvRingtone = findViewById(R.id.tvRingtone);
        swVibrate = findViewById(R.id.swVibrate);
        int[] ids = {R.id.cbMon, R.id.cbTue, R.id.cbWed, R.id.cbThu,
R.id.cbFri, R.id.cbSat, R.id.cbSun};
        for (int i = 0; i < 7; i++) dayBoxes[i] = findViewById(ids[i]);
    }

    private void setupSpinners() {
        spWeekday.setAdapter(new ArrayAdapter<>(this,
                android.R.layout.simple_spinner_dropdown_item,
Alarm.DAY_FULL));
        spRounds.setAdapter(new ArrayAdapter<>(this,
                android.R.layout.simple_spinner_dropdown_item, new String[]
{"1", "2", "3", "4", "5"}));
```

```
    }

    /** Đổ dữ liệu của báo thức lên giao diện. */
    private void fillForm() {
        timePicker.setHour(alarm.hour);
        timePicker.setMinute(alarm.minute);
        edtLabel.setText(alarm.label);

        switch (alarm.repeatType) {
            case Alarm.REPEAT_DAILY:  rgRepeat.check(R.id.rbDaily);  break;
            case Alarm.REPEAT_WEEKLY: rgRepeat.check(R.id.rbWeekly); break;
            case Alarm.REPEAT_CUSTOM: rgRepeat.check(R.id.rbCustom); break;
            default:                  rgRepeat.check(R.id.rbOnce);
        }
        // Hằng tuần: chọn thứ ứng với bit đầu tiên được bật
        int weekday = 0;
        for (int i = 0; i < 7; i++) if ((alarm.daysMask & (1 << i)) != 0) {
weekday = i; break; }
        spWeekday.setSelection(weekday);
        for (int i = 0; i < 7; i++) dayBoxes[i].setChecked((alarm.daysMask &
(1 << i)) != 0);
        updateRepeatPanels();

        tvRingtone.setText(alarm.ringtoneTitle);
        swVibrate.setChecked(alarm.vibrate);
        rgGame.check(alarm.gameType == Alarm.GAME_SEQUENCE ?
R.id.rbGameSequence : R.id.rbGameMath);
        spRounds.setSelection(Math.max(0, Math.min(4, alarm.gameRounds -
1)));
    }

    /** Hiện/ẩn khu vực chọn thứ theo kiểu lặp đang chọn. */
    private void updateRepeatPanels() {
        int checked = rgRepeat.getCheckedRadioButtonId();
        layoutWeekly.setVisibility(checked == R.id.rbWeekly ? View.VISIBLE :
View.GONE);
        layoutDays.setVisibility(checked == R.id.rbCustom ? View.VISIBLE :
View.GONE);
    }

    private void saveAlarm() {
        alarm.hour = timePicker.getHour();
        alarm.minute = timePicker.getMinute();
```

```
        alarm.label = edtLabel.getText().toString().trim();

        int checked = rgRepeat.getCheckedRadioButtonId();
        if (checked == R.id.rbDaily) {
            alarm.repeatType = Alarm.REPEAT_DAILY;
            alarm.daysMask = 0x7F;
        } else if (checked == R.id.rbWeekly) {
            alarm.repeatType = Alarm.REPEAT_WEEKLY;
            alarm.daysMask = 1 << spWeekday.getSelectedItemPosition();
        } else if (checked == R.id.rbCustom) {
            int mask = 0;
            for (int i = 0; i < 7; i++) if (dayBoxes[i].isChecked()) mask |=
(1 << i);
            if (mask == 0) {
                Toast.makeText(this, "Hãy chọn ít nhất một ngày trong tuần",
Toast.LENGTH_SHORT).show();
                return;
            }
            alarm.repeatType = Alarm.REPEAT_CUSTOM;
            alarm.daysMask = mask;
        } else {
            alarm.repeatType = Alarm.REPEAT_ONCE;
            alarm.daysMask = 0;
        }

        alarm.vibrate = swVibrate.isChecked();
        alarm.gameType = rgGame.getCheckedRadioButtonId() ==
R.id.rbGameSequence
                ? Alarm.GAME_SEQUENCE : Alarm.GAME_MATH;
        alarm.gameRounds = spRounds.getSelectedItemPosition() + 1;
        alarm.enabled = true;

        AlarmStore.save(this, alarm);                       // gán id nếu là
báo thức mới
        long at = AlarmScheduler.schedule(this, alarm);
        Toast.makeText(this, AlarmScheduler.timeUntilText(at),
Toast.LENGTH_LONG).show();
        finish();
    }

    // ---------------- Hộp thoại chọn kiểu chuông ----------------

    private void showRingtoneDialog() {
```

```
        List<String> titles = new ArrayList<>();
        List<String> uris = new ArrayList<>();
        titles.add("Mặc định");
        uris.add("");

        RingtoneManager rm = new RingtoneManager(this);
        rm.setType(RingtoneManager.TYPE_ALARM);             // chỉ lấy âm
báo thức của hệ thống
        Cursor c = rm.getCursor();
        while (c.moveToNext()) {
            titles.add(c.getString(RingtoneManager.TITLE_COLUMN_INDEX));
            uris.add(rm.getRingtoneUri(c.getPosition()).toString());
        }

        int checked = Math.max(0, uris.indexOf(alarm.ringtoneUri));
        final int[] selected = {checked};

        new AlertDialog.Builder(this)
                .setTitle("Chọn kiểu chuông")
                .setSingleChoiceItems(titles.toArray(new String[0]), checked,
(d, which) -> {
                    selected[0] = which;
                    playPreview(uris.get(which));           // nghe thử
                })
                .setPositiveButton("Chọn", (d, w) -> {
                    alarm.ringtoneUri = uris.get(selected[0]);
                    alarm.ringtoneTitle = titles.get(selected[0]);
                    tvRingtone.setText(alarm.ringtoneTitle);
                })
                .setNegativeButton("Hủy", null)
                .setOnDismissListener(d -> stopPreview())
                .show();
    }

    private void playPreview(String uriString) {
        stopPreview();
        Uri uri = uriString.isEmpty()
                ? RingtoneManager.getDefaultUri(RingtoneManager.TYPE_ALARM)
                : Uri.parse(uriString);
        previewRingtone = RingtoneManager.getRingtone(this, uri);
        if (previewRingtone != null) previewRingtone.play();
    }
```

```
    private void stopPreview() {
        if (previewRingtone != null && previewRingtone.isPlaying())
previewRingtone.stop();
        previewRingtone = null;
    }

    @Override
    protected void onPause() {
        stopPreview();
        super.onPause();
    }
}
```

Bước 10. Màn hình ③: mini game – `MathQuestion` và `AlarmRingActivity`

`MathQuestion` sinh phép cộng, trừ hoặc nhân ngẫu nhiên kèm 4 đáp án (1 đúng, 3 nhiễu lệch ±1, ±2, ±10… để phải tính thật):

```
package com.example.smartalarm;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Random;

/** Sinh một phép toán ngẫu nhiên kèm 4 đáp án (1 đúng, 3 nhiễu). */
public class MathQuestion {

    private static final Random RANDOM = new Random();

    public final String text;          // ví dụ "47 + 38 = ?"
    public final int answer;
    public final List<Integer> options;

    private MathQuestion(String text, int answer, List<Integer> options) {
        this.text = text;
        this.answer = answer;
        this.options = options;
    }

    public static MathQuestion random() {
        int a, b, answer;
```

```
        String op;
        switch (RANDOM.nextInt(3)) {
            case 0:   // cộng 2 chữ số
                a = 10 + RANDOM.nextInt(90);
                b = 10 + RANDOM.nextInt(90);
                answer = a + b; op = "+";
                break;
            case 1:   // trừ, kết quả không âm
                a = 20 + RANDOM.nextInt(80);
                b = 1 + RANDOM.nextInt(a);
                answer = a - b; op = "−";
                break;
            default:  // nhân 1 chữ số với 2 chữ số
                a = 2 + RANDOM.nextInt(8);
                b = 11 + RANDOM.nextInt(19);
                answer = a * b; op = "×";
        }

        // Tạo 3 đáp án nhiễu gần với đáp án đúng, không trùng nhau
        List<Integer> options = new ArrayList<>();
        options.add(answer);
        int[] deltas = {1, 2, 10, -1, -2, -10, 9, 11};
        while (options.size() < 4) {
            int wrong = answer + deltas[RANDOM.nextInt(deltas.length)];
            if (wrong >= 0 && !options.contains(wrong)) options.add(wrong);
        }
        Collections.shuffle(options);
        return new MathQuestion(a + " " + op + " " + b + " = ?", answer,
options);
    }
}
```

`AlarmRingActivity` hiện trên màn hình khóa, chặn nút Back bằng `OnBackPressedCallback`, **bind** vào `AlarmService` trong `onStart()` và gọi `stopAlarm()` khi thắng. Luật chơi:

- **Giải toán**: trả lời đúng liên tiếp `gameRounds` câu; sai một câu thì tiến độ về 0.
- **Bấm theo thứ tự**: 9 số xáo trộn trong lưới 3×3, chạm lần lượt 1 → 9; chạm sai thì xáo lại vòng đó; hoàn thành `gameRounds` vòng.

```
package com.example.smartalarm;

import android.content.ComponentName;
import android.content.Context;
import android.content.Intent;
import android.content.ServiceConnection;
import android.os.Build;
import android.os.Bundle;
import android.os.IBinder;
import android.view.View;
import android.view.WindowManager;
import android.widget.Button;
import android.widget.GridLayout;
import android.widget.LinearLayout;
import android.widget.TextView;
import android.widget.Toast;

import androidx.activity.OnBackPressedCallback;
import androidx.appcompat.app.AppCompatActivity;

import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

/** Màn hình 3: báo thức đang kêu – phải hoàn thành mini game mới tắt được.
*/
public class AlarmRingActivity extends AppCompatActivity {

    private Alarm alarm;
    private int progress = 0;              // số câu/vòng đã hoàn thành

    // --- Views ---
    private TextView tvProgress, tvQuestion;
    private LinearLayout panelMath, panelSequence;
    private final Button[] answerButtons = new Button[4];
    private GridLayout gridSequence;

    // --- Mini game Giải toán ---
    private MathQuestion question;

    // --- Mini game Bấm theo thứ tự ---
    private int nextNumber = 1;
```

```
    // --- Bound Service ---
    private AlarmService alarmService;
    private boolean bound = false;

    private final ServiceConnection connection = new ServiceConnection() {
        @Override
        public void onServiceConnected(ComponentName name, IBinder binder) {
            alarmService = ((AlarmService.LocalBinder) binder).getService();
            bound = true;
        }

        @Override
        public void onServiceDisconnected(ComponentName name) {
            bound = false;
            alarmService = null;
        }
    };

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        showOverLockScreen();
        setContentView(R.layout.activity_alarm_ring);

        int alarmId = getIntent().getIntExtra(AlarmScheduler.EXTRA_ALARM_ID,
-1);
        alarm = AlarmStore.get(this, alarmId);
        if (alarm == null) alarm = new Alarm();

        ((TextView) findViewById(R.id.tvRingLabel)).setText(
                alarm.label.isEmpty() ? "Báo thức " + alarm.getTimeText() :
alarm.label);
        tvProgress = findViewById(R.id.tvProgress);
        tvQuestion = findViewById(R.id.tvQuestion);
        panelMath = findViewById(R.id.panelMath);
        panelSequence = findViewById(R.id.panelSequence);
        gridSequence = findViewById(R.id.gridSequence);
        answerButtons[0] = findViewById(R.id.btnAnswer1);
        answerButtons[1] = findViewById(R.id.btnAnswer2);
        answerButtons[2] = findViewById(R.id.btnAnswer3);
        answerButtons[3] = findViewById(R.id.btnAnswer4);

        // Chặn nút Back: không thể thoát khi chưa xong mini game
```

```
        getOnBackPressedDispatcher().addCallback(this, new
OnBackPressedCallback(true) {
            @Override
            public void handleOnBackPressed() {
                Toast.makeText(AlarmRingActivity.this,
                        "Hãy hoàn thành mini game để tắt báo thức!",
Toast.LENGTH_SHORT).show();
            }
        });

        if (alarm.gameType == Alarm.GAME_SEQUENCE) {
            panelMath.setVisibility(View.GONE);
            panelSequence.setVisibility(View.VISIBLE);
            newSequenceRound();
        } else {
            panelMath.setVisibility(View.VISIBLE);
            panelSequence.setVisibility(View.GONE);
            for (int i = 0; i < 4; i++) {
                final int index = i;
                answerButtons[i].setOnClickListener(v -> onAnswer(index));
            }
            newMathQuestion();
        }
        updateProgress();
    }

    /** Hiện Activity trên màn hình khóa và bật sáng màn hình. */
    private void showOverLockScreen() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O_MR1) {
            setShowWhenLocked(true);
            setTurnScreenOn(true);
        } else {

getWindow().addFlags(WindowManager.LayoutParams.FLAG_SHOW_WHEN_LOCKED
                    | WindowManager.LayoutParams.FLAG_TURN_SCREEN_ON);
        }
        getWindow().addFlags(WindowManager.LayoutParams.FLAG_KEEP_SCREEN_ON);
    }

    @Override
    protected void onStart() {
        super.onStart();
        bindService(new Intent(this, AlarmService.class), connection,
```

```
Context.BIND_AUTO_CREATE);
    }

    @Override
    protected void onStop() {
        if (bound) {
            unbindService(connection);
            bound = false;
        }
        super.onStop();
    }

    // ================= Mini game 1: Giải toán =================

    private void newMathQuestion() {
        question = MathQuestion.random();
        tvQuestion.setText(question.text);
        for (int i = 0; i < 4; i++) {

answerButtons[i].setText(String.valueOf(question.options.get(i)));
        }
    }

    private void onAnswer(int index) {
        if (question.options.get(index) == question.answer) {
            progress++;
            if (progress >= alarm.gameRounds) { onGameWon(); return; }
            Toast.makeText(this, "Chính xác!", Toast.LENGTH_SHORT).show();
        } else {
            progress = 0;   // sai thì làm lại từ đầu cho tỉnh ngủ
            Toast.makeText(this, "Sai rồi! Đáp án là " + question.answer + ".
Làm lại từ đầu.",
                    Toast.LENGTH_SHORT).show();
        }
        updateProgress();
        newMathQuestion();
    }

    // ================= Mini game 2: Bấm theo thứ tự 1 -> 9
=================

    private void newSequenceRound() {
        nextNumber = 1;
```

```
        List<Integer> numbers = new ArrayList<>();
        for (int i = 1; i <= 9; i++) numbers.add(i);
        Collections.shuffle(numbers);

        for (int i = 0; i < gridSequence.getChildCount(); i++) {
            Button b = (Button) gridSequence.getChildAt(i);
            int value = numbers.get(i);
            b.setText(String.valueOf(value));
            b.setEnabled(true);
            b.setAlpha(1f);
            b.setOnClickListener(v -> onSequenceTap(b, value));
        }
        tvQuestion.setText("Chạm các số từ 1 đến 9 theo thứ tự");
    }

    private void onSequenceTap(Button button, int value) {
        if (value == nextNumber) {
            button.setEnabled(false);
            button.setAlpha(0.3f);
            nextNumber++;
            if (nextNumber > 9) {                // xong một vòng
                progress++;
                if (progress >= alarm.gameRounds) { onGameWon(); return; }
                updateProgress();
                newSequenceRound();
            }
        } else {
            Toast.makeText(this, "Sai thứ tự! Bắt đầu lại vòng này.",
Toast.LENGTH_SHORT).show();
            newSequenceRound();
        }
    }

    // ================= Kết thúc =================

    private void updateProgress() {
        String unit = alarm.gameType == Alarm.GAME_MATH ? "Câu " : "Vòng ";
        tvProgress.setText(unit + (progress + 1) + "/" + alarm.gameRounds);
    }

    private void onGameWon() {
        if (bound && alarmService != null) {
            alarmService.stopAlarm();                                    //
```

```
gọi qua Bound Service
        } else {
            stopService(new Intent(this, AlarmService.class));         //
phương án dự phòng
        }
        Toast.makeText(this, "Đã tắt báo thức. Chúc một ngày tốt lành!",
Toast.LENGTH_LONG).show();
        finish();
    }
}
```

Bước 11. Khai báo `AndroidManifest.xml`

Ghi đè toàn bộ Manifest. Chú ý các điểm gắn với slide: `<service>` có `foregroundServiceType`, hai `<receiver>` khai báo tĩnh, `<permission>` tự định nghĩa với `protectionLevel="signature"` để bảo vệ broadcast nội bộ.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">

    <!-- Permission tự định nghĩa để bảo vệ broadcast nội bộ (chỉ app cùng
chữ ký mới dùng được) -->
    <permission
        android:name="com.example.smartalarm.permission.INTERNAL_BROADCAST"
        android:protectionLevel="signature" />
    <uses-permission
android:name="com.example.smartalarm.permission.INTERNAL_BROADCAST" />

    <!-- Thông báo (Android 13+ phải xin lúc chạy) -->
    <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
    <!-- Báo thức chính xác: Android 12-12L dùng SCHEDULE_EXACT_ALARM,
Android 13+ dùng USE_EXACT_ALARM -->
    <uses-permission
        android:name="android.permission.SCHEDULE_EXACT_ALARM"
        android:maxSdkVersion="32" />
    <uses-permission android:name="android.permission.USE_EXACT_ALARM" />
    <!-- Foreground Service phát âm thanh -->
    <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
    <uses-permission
android:name="android.permission.FOREGROUND_SERVICE_MEDIA_PLAYBACK" />
    <!-- Hiện Activity toàn màn hình khi khóa máy -->
    <uses-permission android:name="android.permission.USE_FULL_SCREEN_INTENT"
```

```
/>
    <uses-permission android:name="android.permission.VIBRATE" />
    <uses-permission android:name="android.permission.WAKE_LOCK" />
    <!-- Nhận broadcast khởi động xong -->
    <uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"
/>

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="SmartAlarm"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.SmartAlarm">

        <activity
            android:name=".MainActivity"
            android:exported="true"
            android:label="Báo thức">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity
            android:name=".AddEditAlarmActivity"
            android:exported="false"
            android:parentActivityName=".MainActivity" />

        <activity
            android:name=".AlarmRingActivity"
            android:excludeFromRecents="true"
            android:exported="false"
            android:launchMode="singleTask"
            android:showWhenLocked="true"
            android:taskAffinity=""
            android:theme="@style/Theme.SmartAlarm.Ring"
            android:turnScreenOn="true" />

        <service
            android:name=".AlarmService"
            android:exported="false"
```

```
            android:foregroundServiceType="mediaPlayback" />

        <!-- Receiver tĩnh: nhận Intent từ AlarmManager -->
        <receiver
            android:name=".AlarmReceiver"
            android:exported="false" />

        <!-- Receiver tĩnh: nhận broadcast hệ thống khi máy khởi động xong --
>
        <receiver
            android:name=".BootReceiver"
            android:exported="false">
            <intent-filter>
                <action android:name="android.intent.action.BOOT_COMPLETED"
/>
            </intent-filter>
        </receiver>
    </application>
</manifest>
```

Quyền	Vì sao cần
`POST_NOTIFICATIONS`	Android 13+ phải xin lúc chạy mới hiện được Notification
`SCHEDULE_EXACT_ALARM` / `USE_EXACT_ALARM`	Báo thức chính xác bằng `setAlarmClock()` từ Android 12
`FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_MEDIA_PLAYBACK`	Chạy Foreground Service loại phát âm thanh (bắt buộc từ Android 14)
`USE_FULL_SCREEN_INTENT`	Mở màn hình mini game khi máy đang khóa
`VIBRATE`, `WAKE_LOCK`	Rung và giữ máy thức khi chuông reo
`RECEIVE_BOOT_COMPLETED`	`BootReceiver` đặt lại báo thức sau khi khởi động lại máy

5. Chạy thử và kiểm tra

Chạy trên AVD Android 13 hoặc 14 (API 33–34) có Google Play. Đặt báo thức sớm hơn giờ hiện tại 1–2 phút để không phải chờ lâu; lần đầu mở app hãy bấm **Cho phép** quyền thông báo.

5.1. Kịch bản kiểm tra

#	Thao tác	Kết quả mong đợi
1	Thêm báo thức Một lần, sau 1 phút, game Giải toán 2 câu; bấm Lưu	Toast “Báo thức sẽ kêu sau 1 phút”; thẻ mới xuất hiện; biểu tượng đồng hồ trên thanh trạng thái
2	Khóa màn hình (nút nguồn), chờ đến giờ	Màn hình sáng, hiện màn hình ③ trên màn hình khóa, chuông kêu liên tục, máy rung
3	Để máy mở khóa khi đến giờ	Hiện Notification dạng heads-up; chạm hoặc bấm “Giải mini game” → mở màn hình ③
4	Trên màn hình ③ bấm Back; vuốt Notification	Toast nhắc làm mini game; Notification không bị xóa; chuông vẫn kêu
5	Trả lời sai 1 câu	Toast báo đáp án đúng, tiến độ về “Câu 1/2”
6	Trả lời đúng đủ 2 câu	Chuông tắt, Notification biến mất, màn hình đóng; báo thức Một lần chuyển sang trạng thái tắt (mờ) trong danh sách
7	Báo thức game Bấm 1→9, chạm sai thứ tự	Toast “Sai thứ tự”, lưới xáo lại
8	Báo thức Hằng ngày, sau khi tắt chuông	Thẻ vẫn bật; `adb shell dumpsys alarm` cho thấy lịch của ngày hôm sau
9	Báo thức Hằng tuần vào thứ hôm nay, giờ đã qua	Toast “sau 6 ngày …” (kêu vào thứ này tuần sau)
10	Tùy chọn: bỏ chọn hết 7 ngày rồi Lưu	Toast “Hãy chọn ít nhất một ngày”, không lưu
11	Chọn chuông Oxygen trong hộp thoại	Nghe thử khi chạm; hết tiếng khi đóng hộp thoại; lúc báo thức kêu đúng chuông đã chọn

#	Thao tác	Kết quả mong đợi
12	Đặt báo thức sau 3 phút rồi chạy `adb reboot`	Sau khi máy khởi động lại, báo thức vẫn kêu đúng giờ (BootReceiver)
13	Nhấn giữ một thẻ → Xóa	Thẻ biến mất, báo thức đó không còn kêu

Lệnh hữu ích khi kiểm tra (Terminal của Android Studio):

```
# Xem các báo thức đang được AlarmManager giữ cho ứng dụng
adb shell dumpsys alarm | grep -A3 com.example.smartalarm

# Giả lập chế độ ngủ sâu (Doze) để thử setAlarmClock vẫn kêu
adb shell dumpsys deviceidle force-idle

# Khởi động lại máy ảo để thử BootReceiver
adb reboot
```

5.2. Lỗi thường gặp

Hiện tượng	Nguyên nhân	Cách sửa
Không hiện Notification, không hiện màn hình ③	Chưa cấp quyền thông báo (Android 13+)	Cài đặt → Ứng dụng → SmartAlarm → Thông báo → Bật
Crash `ForegroundServiceStartNotAllowedException` hoặc `MissingForegroundServiceTypeException`	Thiếu `foregroundServiceType` hoặc quyền `FOREGROUND_SERVICE_MEDIA_PLAYBACK`	Kiểm tra lại Manifest ở Bước 11
Crash `RemoteServiceException: Did not then call Service.startForeground()`	`startForeground()` gọi quá trễ hoặc bị bỏ qua	Gọi `startForeground()` ngay đầu `onStartCommand()`, trước mọi xử lý nặng
Crash `SecurityException` khi gọi `setAlarmClock()`	Thiếu quyền báo thức chính xác	Khai báo `USE_EXACT_ALARM`; trên Android 12 mở Cài đặt → Báo thức & lời nhắc
Khi máy đang mở khóa, màn hình ③ không tự bật	Hành vi đúng của Android: full-screen intent chỉ tự mở khi màn hình tắt/khóa	Chạm vào Notification heads-up; muốn thử full-screen thì khóa máy trước
Android 14: màn hình ③ không hiện trên màn hình khóa	Quyền full-screen intent bị tắt	Bấm "Mở cài đặt" trong hộp thoại của `MainActivity` và bật quyền
Không nghe tiếng chuông trên máy ảo	Âm lượng *Báo thức* đang bằng 0	Tăng âm lượng báo thức trong Cài đặt → Âm thanh
Hộp thoại chọn chuông chỉ có "Mặc định"	Ảnh hệ thống của AVD không có âm báo thức	Dùng AVD có Google Play hoặc máy thật
Giao diện bị chè lên thanh trạng thái	`targetSdk = 35` trên Android 15 ép chế độ edge-to-edge	Đặt `targetSdk = 34` như Bước 1

6. Bài tập mở rộng, câu hỏi ôn tập và đánh giá

6.1. Bài tập mở rộng

1. **Trả lời ngay trong Notification (RemoteInput – slide 3.2).** Thêm action "Trả lời nhanh" hiển thị phép toán trong Notification. Tạo `RemoteInput` với key `"answer"`, gắn vào `NotificationCompat.Action` có `PendingIntent.FLAG_MUTABLE`; viết `AnswerReceiver` đọc `RemoteInput.getResultsFromIntent(intent)`, nếu đúng thì gọi `startService()` với action `ACTION_STOP` (tự bổ sung nhánh xử lý này trong onStartCommand()) để tắt chuông.
2. **Báo lại (Snooze) có giới hạn.** Thêm nút "Báo lại 5 phút" trên màn hình ③, tối đa 2 lần cho mỗi lần kêu; dùng `setAlarmClock()` với `requestCode` riêng để không đè lịch gốc.

3. **Tự tắt sau 10 phút và tăng dần âm lượng.** Trong `AlarmService` dùng `Handler.postDelayed()` để tăng `player.setVolume()` mỗi 5 giây và gọi `stopAlarm()` sau 10 phút, đồng thời hiện Notification "Bạn đã bỏ lỡ báo thức".
4. **Mini game thứ ba – Lắc máy.** Dùng `SensorManager` + cảm biến gia tốc; tắt chuông khi lắc đủ 30 lần. Thêm lựa chọn vào `rgGame`.
5. **Độ khó mini game.** Thêm lựa chọn Dễ/Khó: mức Khó dùng phép nhân 2 chữ số × 2 chữ số và đáp án nhập bằng bàn phím số thay vì 4 lựa chọn.
6. **Chuông từ file nhạc của máy.** Mở bộ chọn của hệ thống bằng `RingtoneManager.ACTION_RINGTONE_PICKER` hoặc `ACTION_OPEN_DOCUMENT` (audio/*) và lưu URI.
7. **Chuyển lưu trữ sang SQLite (bài học sau).** Thay `AlarmStore` bằng `SQLiteOpenHelper` với bảng `alarms`; giữ nguyên các phương thức `getAll/get/save/delete` để các lớp khác không phải sửa.
8. **Thống kê dậy sớm.** Lưu thời gian từ lúc chuông reo đến lúc thăng mini game; hiển thị trung bình 7 ngày gần nhất trên màn hình ①.

6.2. Câu hỏi ôn tập

1. Vì sao phát chuông bằng Foreground Service mà không phát trực tiếp trong `AlarmReceiver.onReceive()`?
2. `AlarmService` vừa được start vừa được bind. Khi `stopAlarm()` gọi `stopSelf()` mà `AlarmRingActivity` vẫn còn bind, service có bị hủy ngay không? Khi nào `onDestroy()` được gọi?
3. Nêu khác biệt giữa `AlarmReceiver` (khai báo tĩnh) và receiver trong `MainActivity` (khai báo động). Điều gì xảy ra nếu quên `unregisterReceiver()`?
4. Permission `INTERNAL_BROADCAST` với `protectionLevel="signature"` ngăn được tình huống tấn công nào?
5. Vì sao mỗi báo thức phải dùng `requestCode` khác nhau khi tạo `PendingIntent`? Thử dùng chung `requestCode = 0` cho mọi báo thức và mô tả lỗi quan sát được.
6. Tính tay `daysMask` cho báo thức T2, T4, T6. Hàm `getNextTriggerTime()` trả về gì nếu bây giờ là T6 lúc 07:00 và báo thức đặt 06:30?
7. Vì sao báo thức lặp chỉ đặt **một** lần kêu kế tiếp, rồi đến giờ mới đặt lần sau, thay vì dùng `setRepeating()`?
8. Notification tùy biến bằng `RemoteViews` có những giới hạn gì so với layout của Activity?

6.3. Gợi ý thang điểm (10 điểm)

Nội dung	Điểm
Giao diện 3 màn hình đúng thiết kế, hoạt động thêm/sửa/xóa/bật tắt	2,0
Bốn kiểu lặp và chọn kiểu chuông hoạt động đúng	2,0
AlarmManager + Broadcast Receiver (tĩnh, động, BOOT_COMPLETED)	2,0
Foreground Service phát chuông/rung + Notification custom layout	2,0
Mini game tắt báo thức qua Bound Service, chặn Back	1,5
Trả lời câu hỏi ôn tập khi vấn đáp	0,5
Thưởng: mỗi bài tập mở rộng hoàn thành	+0,5 (tối đa 10)